UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Nombre: Llumiquinga Lema Angel Santiago

Departamento: Eléctrica, Electrónica y Telecomunicaciones

NRC: 36466

Carrera: Telecomunicaciones

Tema: Ejercicios de programación y algoritmia

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

Análisis del problema.

Entrada: Dos números reales.

Proceso: Sumar ambos números y dividirlos para dos.

Salida: Un mensaje con el resultado del promedio.

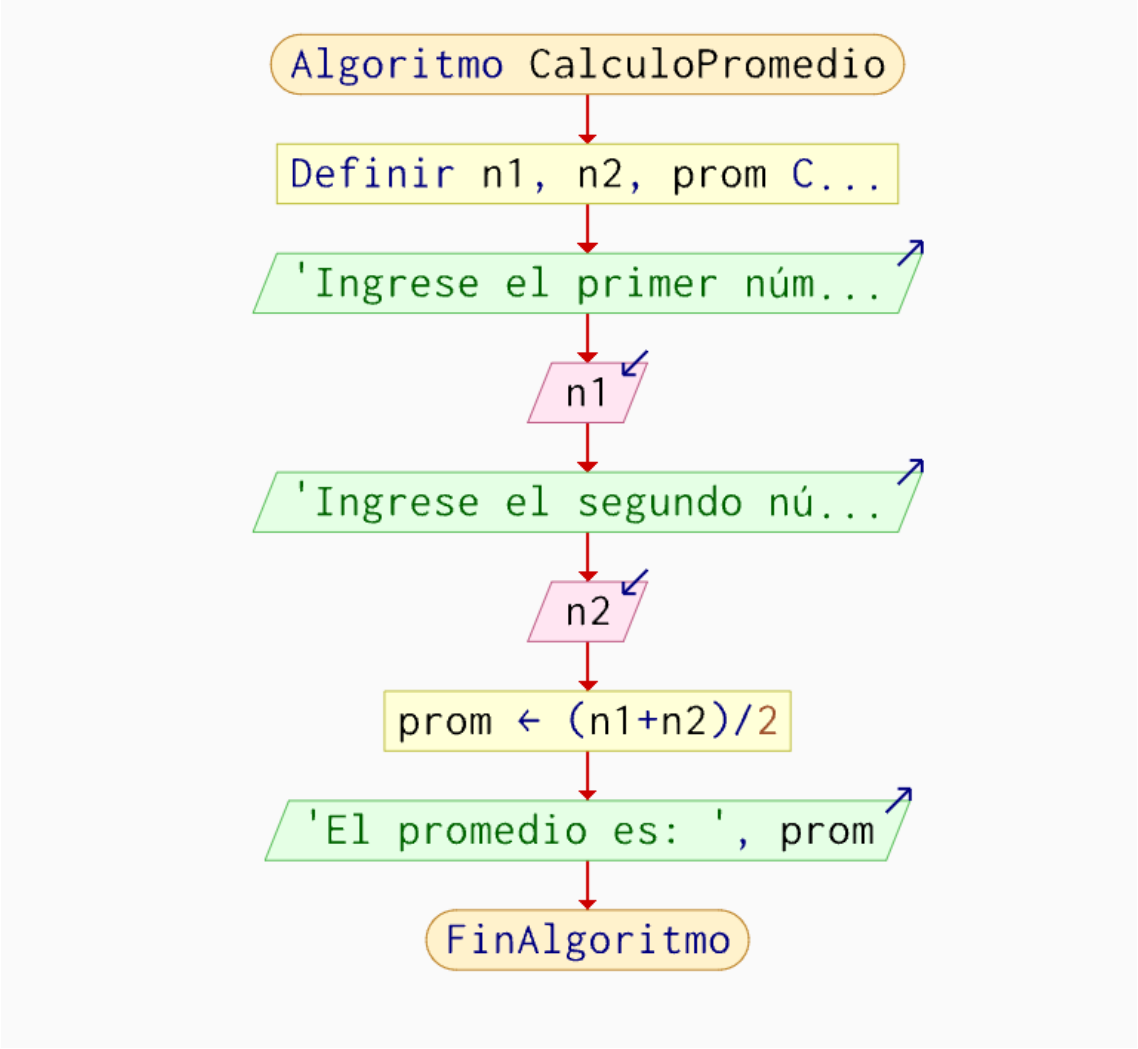
Tabla de Objetos

OBJETO	NOMBRE	VALOR	TIPO
Dato de entrada	n1	Var	Real
Dato de entrada	n2	Var	Real
Dato de salida	prom	Var	Real

Seudocódigo

```
Algoritmo CalculoPromedio
    Definir n1, n2, prom Como Real;
    Escribir "Ingrese el primer número";
    Leer n1;
    Escribir "Ingrese el segundo número";
    Leer n2;
    prom ← (n1 + n2) / 2;
    Escribir "El promedio es: ", prom;
FinAlgoritmo
```

Diagrama de flujo



Prueba de Escritorio

Paso	n1	n2	Prom(n1+n2/2)	Salida
1	4			Ingrese el primer número
2		9		Ingrese el segundo número
3			6.5	
4				El promedio es 6.5

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables de tipo decimal (Real -> float)
    float n1, n2, prom;

    // Lectura del primer número
    printf("Ingrese el primer numero: ");
    scanf("%f", &n1);

    // Lectura del segundo número
    printf("Ingrese el segundo numero: ");
    scanf("%f", &n2);

    // Proceso: Cálculo del promedio
    prom = (n1 + n2) / 2.0;

    // Salida: Mostrar el resultado final
    printf("El promedio es: %.2f\n", prom);

    return 0;
}
```

Ejecución

```
Ingrese el primer numero: 45
Ingrese el segundo numero: 60
El promedio es: 52.50

Process returned 0 (0x0)   execution time : 5.033 s
Press any key to continue.
```

Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

Análisis del problema

Entrada: Un número real.

Proceso: Aplicamos la función abs a la variable y posteriormente elevamos al cubo.

Salida: Mensaje con el resultado del procedimiento

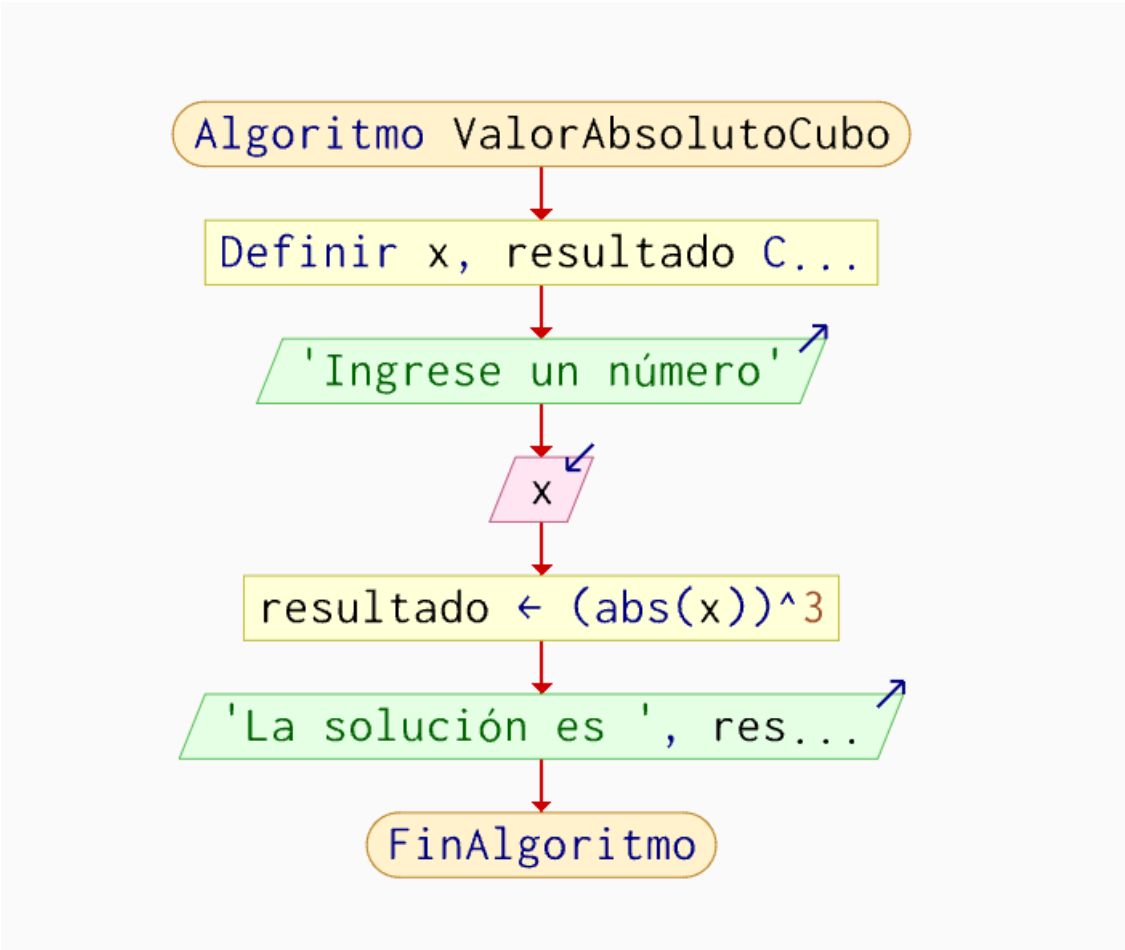
Tabla de Objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	Var	Real
Dato de salida	resultado	Var	Real

Seudocódigo

```
1  Algoritmo ValorAbsolutoCubo
2      Definir x, resultado Como Real;
3      Escribir "Ingrese un número";
4      Leer x;
5      resultado ← (abs(x))↑3;
6      Escribir "La solución es ", resultado;
7  FinAlgoritmo
```

Diagrama de flujo



Prueba de Escritorio

Paso	x	Resultado (abs(x))^3	Salida
1	-6		Ingrese un numero
2		216	
3			La solución es: 216

Código Fuente

```c
#include <stdio.h>

int main()
{
    float x, x_abs, resultado;

    printf("Ingrese un numero: ");
    scanf("%f", &x);

    if (x < 0) {
        x_abs = -x;  // Si es negativo, le invertimos el signo para volverlo positivo
    } else {
        x_abs = x;   // Si es cero o positivo, se queda exactamente igual
    }

    // 2. Lógica propia para elevar al cubo
    resultado = x_abs * x_abs * x_abs;

    // Salida de datos
    printf("La solucion es: %.2f\n", resultado);

    return 0;
}
```

Ejecución

```
Ingrese un numero: -6
La solucion es: 216.00

Process returned 0 (0x0)   execution time : 2.617 s
Press any key to continue.
```

Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

Análisis del problema

Entrada: Dos números enteros.

Proceso: División entre los dos números.

Salida: Mensaje con la división exacta.

Restricción: El denominador debe ser diferente de cero porque no existe división para cero, y su modulo debe ser cero para que sea exacta.

Tabla de Objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n1	Var	Entero
Dato de entrada	n2	Var	Entero
Dato de salida	Resultado	Var	Entero

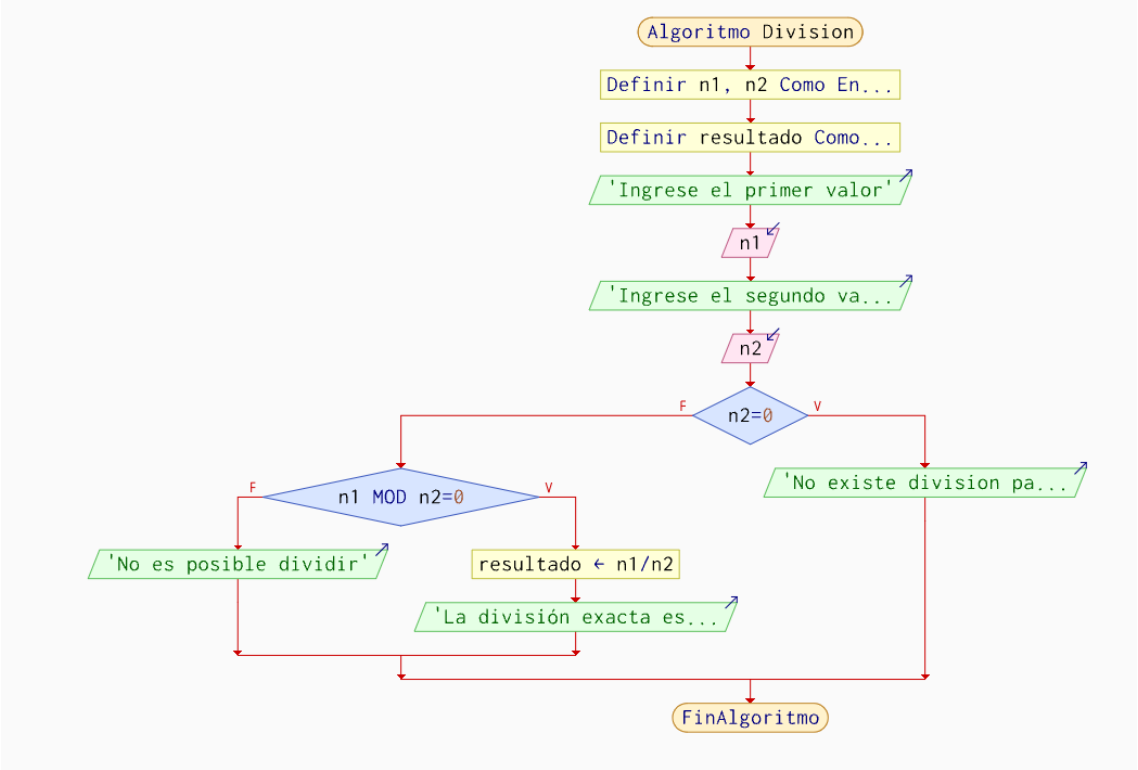
Seudocódigo

```
Algoritmo Division
    Definir n1, n2 Como Entero;
    Definir resultado Como Real;

    Escribir "Ingrese el primer valor";
    Leer n1;
    Escribir "Ingrese el segundo valor";
    Leer n2;

    Si n2 = 0 Entonces
        Escribir "No existe division para cero";
    SiNo
        Si n1 mod n2 = 0 Entonces
            resultado ← n1 / n2;
            Escribir "La división exacta es ", resultado;
        SiNo
            Escribir "No es posible dividir";
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

Paso	n1	n2	Condicion n2=0	Condicion n1MODn2=0	resultado	Salida
1	12					Ingrese el primer valor
2		4	Falso			Ingrese el segundo valor
3				Verdadero		
4					3	
5						La división exacta es 3

Código Fuente

```c

int main()
{
    // Declaración de variables
    int n1, n2;
    float resultado;

    // Entrada de datos
    printf("Ingrese el primer valor: ");
    scanf("%d", &n1);

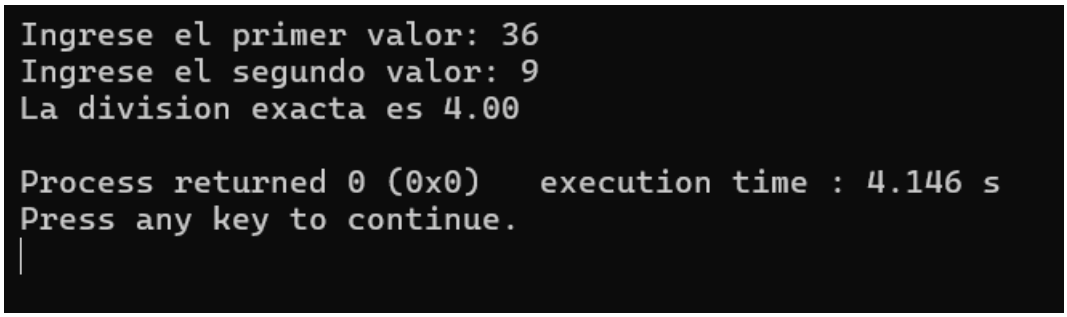
    printf("Ingrese el segundo valor: ");
    scanf("%d", &n2);

    // Evaluación condicional anidada
    if (n2 == 0) {
        printf("No existe division para cero\n");
    } else {

        if (n1 % n2 == 0) {
            // Conversión explícita (float) para asegurar división real limpia
            resultado = (float)n1 / n2;
            printf("La division exacta es %.2f\n", resultado);
        } else {
            printf("No es posible dividir\n");
        }
    }

    return 0;
}
```

Ejecución



Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

Análisis del problema

Entrada: Número real.

Proceso: Evaluar si el numero es mayor o igual a cero y si es menor o igual a 10.

Salida: Indicar si pertenece al conjunto.

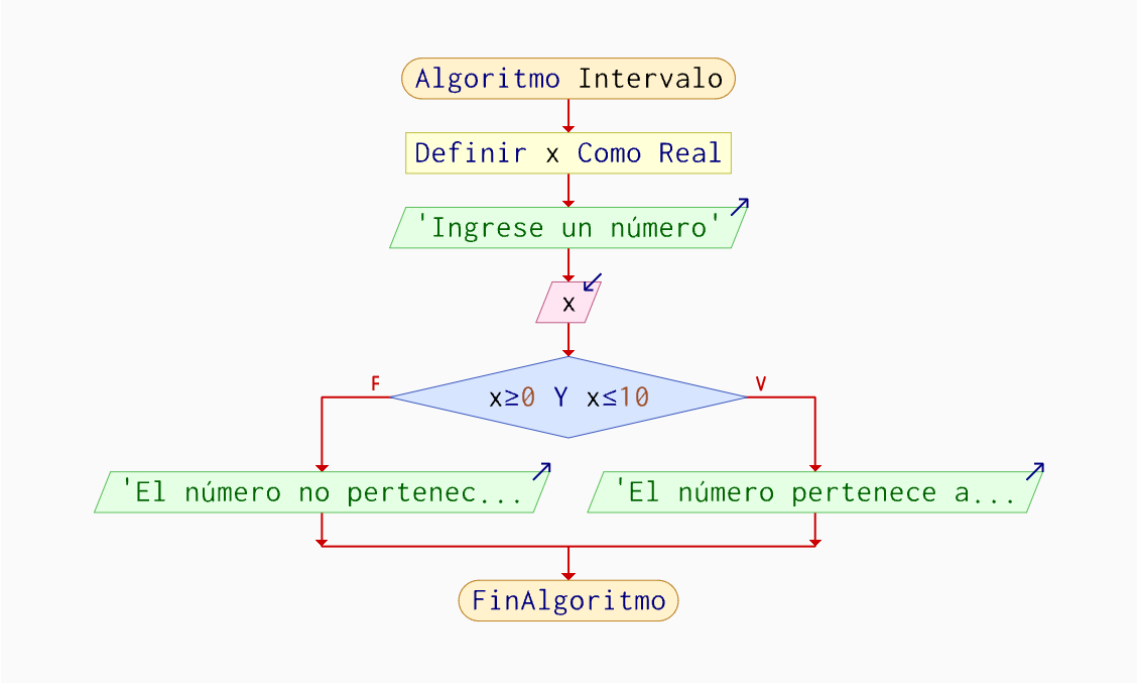
Tabla de objetos

Objetos	Nombre	Valor	Tipo
Dato de entrada	X	Var	Real

Seudocódigo

```
1  Algoritmo Intervalo
2      Definir x Como Real;
3      Escribir "Ingrese un número";
4      Leer x;
5      Si x ≥ 0 Y x ≤ 10 Entonces
6          Escribir "El número pertenece al intervalo [0, 10]";
7      SiNo
8          Escribir "El número no pertenece al intervalo [0, 10]";
9      FinSi
10 FinAlgoritmo
```

Diagrama de flujo



Prueba de Escritorio

Paso	X	Condición(x>=0 Y x<=10)	Salida
1	5		Ingrese un numero
2		Verdadero	
3			El numero pertenece al intervalo [0,10]

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variable de tipo decimal/real
    float x;

    // Lectura del número por teclado
    printf("Ingrese un numero: ");
    scanf("%f", &x);

    // Condicional compuesto utilizando el operador lógico AND (&&)
    if (x >= 0 && x <= 10) {
        printf("El numero pertenece al intervalo [0, 10]\n");
    } else {
        printf("El numero no pertenece al intervalo [0, 10]\n");
    }

    return 0;
}
```

Ejecución

```
Ingrese un numero: 8
El numero pertenece al intervalo [0, 10]

Process returned 0 (0x0)   execution time : 2.526 s
Press any key to continue.
```

Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

Análisis del problema

Entrada: Un número entero.

Proceso: Se realiza una conversión para cada variable.

Salida: Se da un mensaje indicando el valor de cada variable en base al valor ingresado.

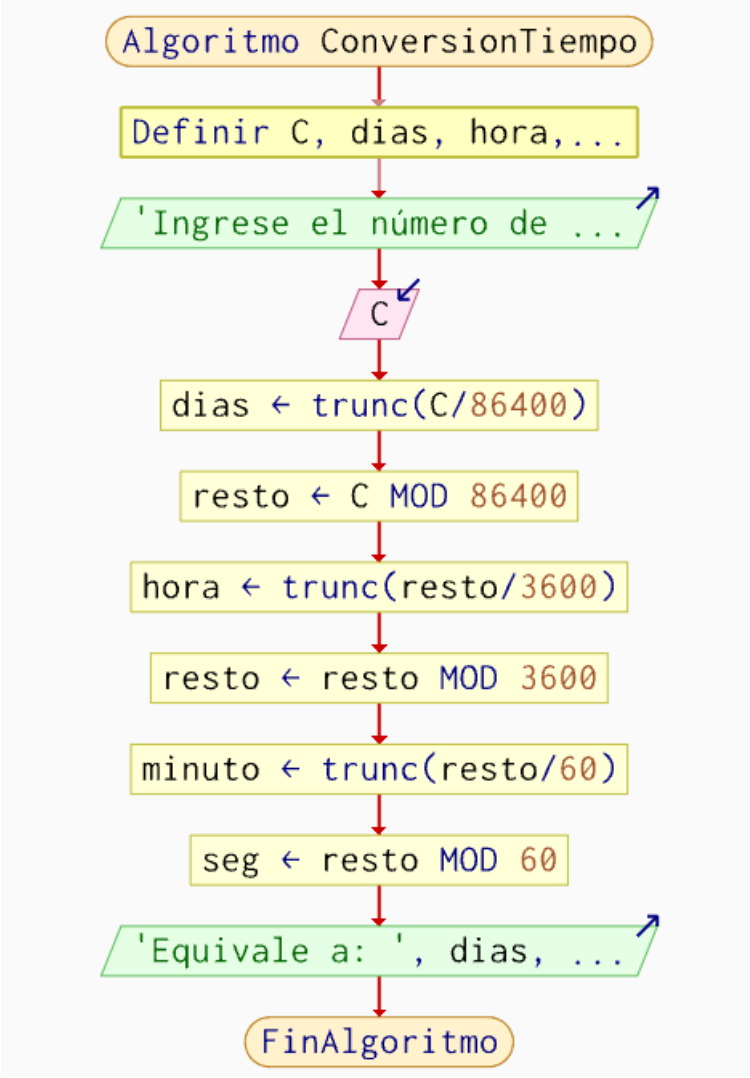
Tabla de Objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	C	Var	Entero
Dato de salida	dias	Var	Entero
Dato de salida	hora	Var	Entero
Dato de salida	minuto	Var	Entero
Dato de salida	seg	Var	Entero

Seudocódigo

```
Algoritmo ConversionTiempo
    Definir C, dias, hora, minuto, seg Como Entero;
    Escribir "Ingrese el número de segundos ";
    Leer C;
    dias ← trunc(C / 86400);
    resto ← C MOD 86400;
    hora ← trunc(resto / 3600);
    resto ← resto MOD 3600;
    minuto ← trunc(resto / 60);
    seg ← resto MOD 60;
    Escribir "Equivale a: ", dias, " dias, ", hora, " horas, ", minuto, " minutos y ", seg, " segundos";
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

c	dias	resto	hora	Minuto	seg	Operación	Salida
90061						Trunc(90061/86400)	Ingrese el numero de segundos
	1					90061 MOD 86400	
		3661				Trunc(3661/3600)	
			1			3661 MOD 3600	
		61		1		Trunc(61/60)	
					1	61 MOD 60	

							Equivale a: 1dia, 1 hora, 1 minuto y 1 segundo

Código fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables tipo entero (int)
    int C, dias, hora, minuto, seg, resto;

    // Entrada de datos
    printf("Ingrese el numero de segundos: ");
    scanf("%d", &C);

    // Proceso: Conversión de segundos a días, horas, minutos y segundos
    // En C, la división de enteros (int / int) trunca automáticamente la parte decimal
    dias = C / 86400;
    resto = C % 86400;  // Operador módulo en C es %

    hora = resto / 3600;
    resto = resto % 3600;

    minuto = resto / 60;
    seg = resto % 60;

    // Salida de datos
    printf("Equivale a: %d dias, %d horas, %d minutos y %d segundos\n", dias, hora, minuto, seg);

    return 0;
}
```

Ejecución

```
Ingrese el numero de segundos: 90061
Equivale a: 1 dias, 1 horas, 1 minutos y 1 segundos

Process returned 0 (0x0)   execution time : 2.918 s
Press any key to continue.
```

Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

Análisis del problema

Entrada: Secuencia indeterminada de números reales hasta que se ingrese un valor negativo.

Proceso: usar un ciclo repetitivo mientras la altura sea positiva. Sumar las alturas y contar cuántas se ingresaron. Comparar para identificar la altura máxima y mínima. Después del ciclo calcular la media.

Salida: imprimir la media el máximo y mínimo.

Tabla de Objetos

Objeto	**Nombre**	**Valor**	**Tipo**
Altura ingresada	altura	Var	real
Suma de altura	suma	Var	real
Cantidad de alturas	contador	Var	entero
Altura máxima	maximo	Var	real
Altura mínima	minimo	Var	real
promedio	media	var	real

Seudocódigo

```
Algoritmo Alturas
    Definir altura, suma, maximo, minimo, media Como Real;
    Definir contador Como Entero;

    suma ← 0;
    contador ← 0;

    Escribir "Ingrese una altura (o un negativo para terminar):";
    Leer altura;

    Si altura ≥ 0 Entonces
        maximo ← altura;
        minimo ← altura;

        Mientras altura ≥ 0 Hacer
            suma ← suma + altura;
            contador ← contador + 1;

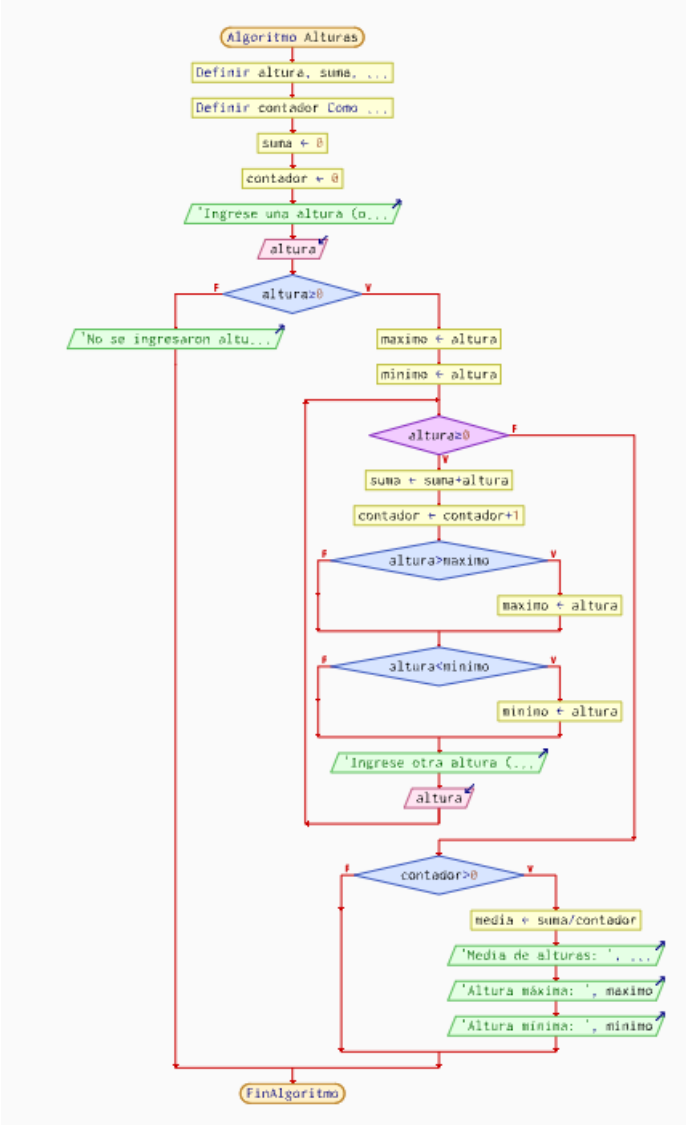
            Si altura > maximo Entonces
                maximo ← altura;
            FinSi

            Si altura < minimo Entonces
                minimo ← altura;
            FinSi

            Escribir "Ingrese otra altura (o negativo para terminar):";
            Leer altura;
        FinMientras

        Si contador > 0 Entonces
            media ← suma / contador;
            Escribir "Media de alturas: ", media;
            Escribir "Altura máxima: ", maximo;
            Escribir "Altura mínima: ", minimo;
        FinSi
    SiNo
        Escribir "No se ingresaron alturas válidas.";
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

Altura	Suma	Contador	Máximo	Mínimo	Media	Condición altura>=0	salida
1.70	1.70	1	1.70	1.70		verdadero	Ingrese una altura
1.80	3.50	2	1.80	1.70		Verdadero	Ingrese otra altura
1.60	5.10	3	1.80	1.60		verdadero	Ingrese otra altura

-1	5.10	3	1.80	1.60		Falso	Ingrese otra altura
					1.70		Media:1.70 Maximo1.80 minimo:1.60

Código fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables
    float altura, suma = 0.0, maximo, minimo, media;
    int contador = 0;

    // Entrada del primer valor
    printf("Ingrese una altura (o un negativo para terminar): ");
    scanf("%f", &altura);

    // Verificación inicial si el primer número es válido
    if (altura >= 0) {
        maximo = altura;
        minimo = altura;

        // Bucle para procesar alturas hasta ingresar un número negativo
        while (altura >= 0) {
            suma = suma + altura;
            contador = contador + 1;

            // Actualización del valor máximo
            if (altura > maximo) {
                maximo = altura;
            }

            // Actualización del valor mínimo
            if (altura < minimo) {
                minimo = altura;
            }

            printf("Ingrese otra altura (o negativo para terminar): ");
            scanf("%f", &altura);
        }

        // Cálculo y despliegue de resultados si se ingresó al menos un dato válido
        if (contador > 0) {
            media = suma / contador;
            printf("\n--- RESULTADOS ---\n");
            printf("Media de alturas: %.2f\n", media);
            printf("Altura maxima: %.2f\n", maximo);
            printf("Altura minima: %.2f\n", minimo);
        }
    } else {
        printf("No se ingresaron alturas validas.\n");
    }

    return 0;
}
```

Ejecución

```
Ingrese una altura (o un negativo para terminar): 1.70
Ingrese otra altura (o negativo para terminar): 1.80
Ingrese otra altura (o negativo para terminar): 1.60
Ingrese otra altura (o negativo para terminar): -1

--- RESULTADOS ---
Media de alturas: 1.70
Altura maxima: 1.80
Altura minima: 1.60

Process returned 0 (0x0)   execution time : 15.217 s
Press any key to continue.
```

Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

Análisis del problema

Entrada: Un número positivo entero.

Proceso: iniciar un contador en 1, iterar en un bucle incrementando en 1 cada vuelta al terminar el ciclo calcular la raíz cuadrada entera por defecto.

Salida: el valor entero de la raíz.

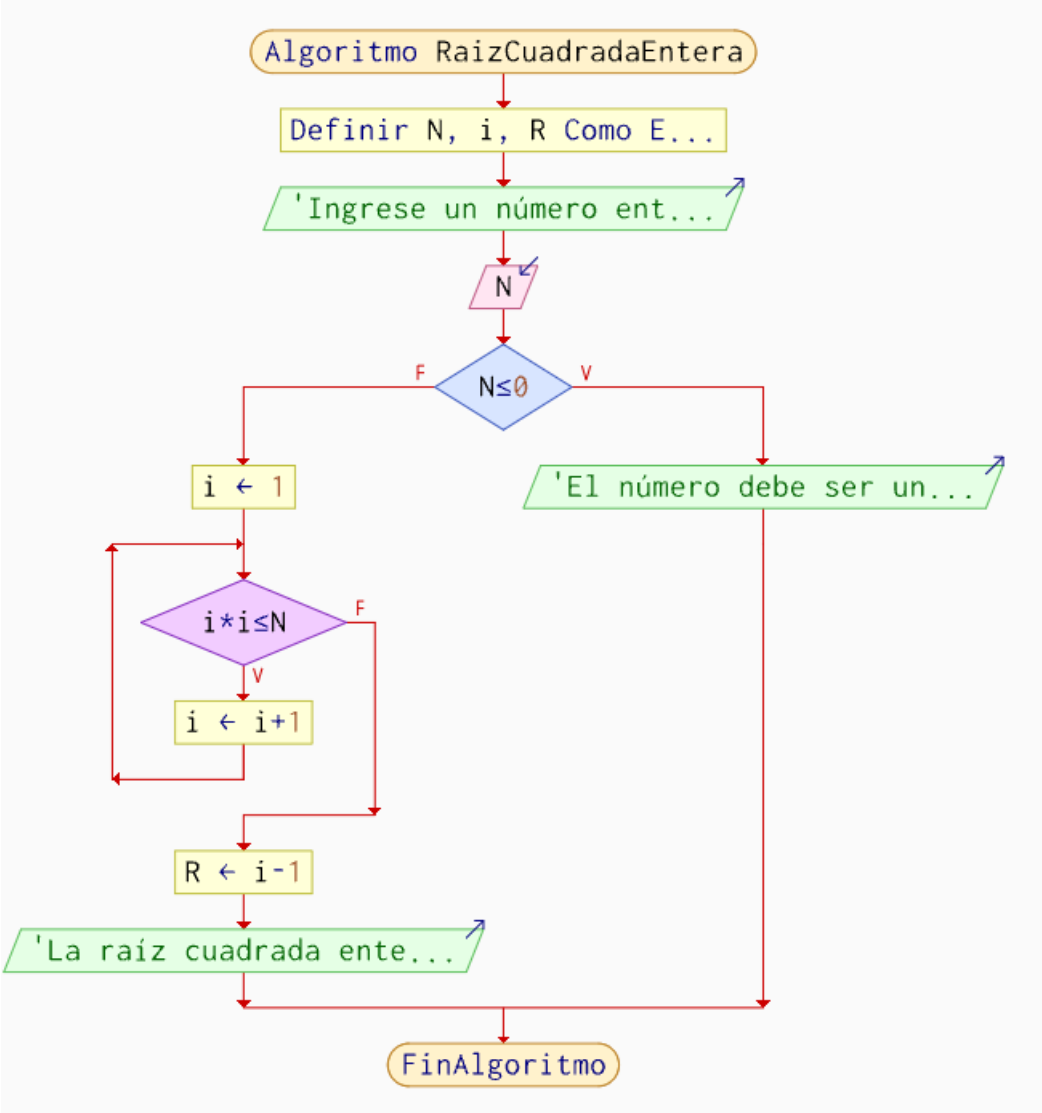
Tabla de objetos

Objeto	**Nombre**	**Valor**	**Tipo**
Numero de entrada	N	Var	Entero
Contador	i	Var	Entero
Raíz entera	R	Var	Entero

Seudocódigo

```
Algoritmo RaizCuadradaEntera
    Definir N, i, R Como Entero;
    Escribir "Ingrese un número entero positivo N:";
    Leer N;
    Si N ≤ 0 Entonces
        Escribir "El número debe ser un entero positivo mayor que cero.";
    SiNo
        i ← 1;
        Mientras i * i ≤ N Hacer
            i ← i + 1;
        FinMientras
        R ← i - 1;
        Escribir "La raíz cuadrada entera por defecto de ", N, " es: ", R;
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

R	i	N	Condición (N>=0)(i*i<=N)	Salida
		10	Falso	Ingrese un numero entero mayor que cero
	1		Verdadero	
	2		Verdadero	
	3		Verdadero	
	4		Falso	
3				La raiz cuadrada entera por defecto es 3

Código fuente

```c
int main()
{
    // Declaración de variables enteras
    int N, i, R;

    // Lectura de datos por teclado
    printf("Ingrese un numero entero positivo N: ");
    scanf("%d", &N);

    // Validación inicial
    if (N <= 0) {
        printf("El numero debe ser un entero positivo mayor que cero.\n");
    } else {
        i = 1;

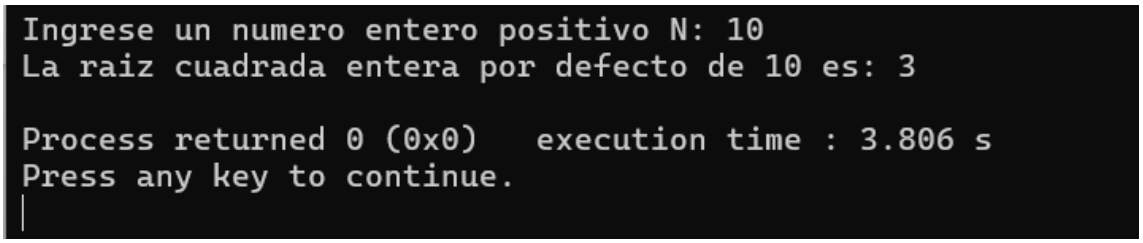
        // Bucle para buscar la raíz entera mediante multiplicaciones
        while (i * i <= N) {
            i = i + 1;
        }

        // Como 'i' se pasa por una unidad al romper el bucle, restamos 1
        R = i - 1;

        // Salida de datos
        printf("La raiz cuadrada entera por defecto de %d es: %d\n", N, R);
    }

    return 0;
}
```

Ejecución



Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

Análisis del problema

entrada: Un número entero que sea mayor a 1.

Proceso: Validar si el número es mayor que 1 inicializar un contador de divisiones y un iterador. Recorrer un bucle desde 1 hasta el número en cada vuelta evaluar si su módulo es cero. Si es cero, incrementar el contador y evaluar la condición final.

Salida: Un mensaje indicando si el número es primo o no.

Tabla de objetos

Objeto	**Nombre**	**Valor**	**tipo**
Numero a evaluar	**n**	**Var**	**entero**
Iterador	**i**	**Var**	**entero**
Contador	**divisores**	**Var**	**entero**
Estado de divisibilidad	**Primo**	**Var**	**lógico**

Seudocódigo

```
Algoritmo NumeroPrimo
    Definir n, i Como Entero;
    Definir primo Como Logico;

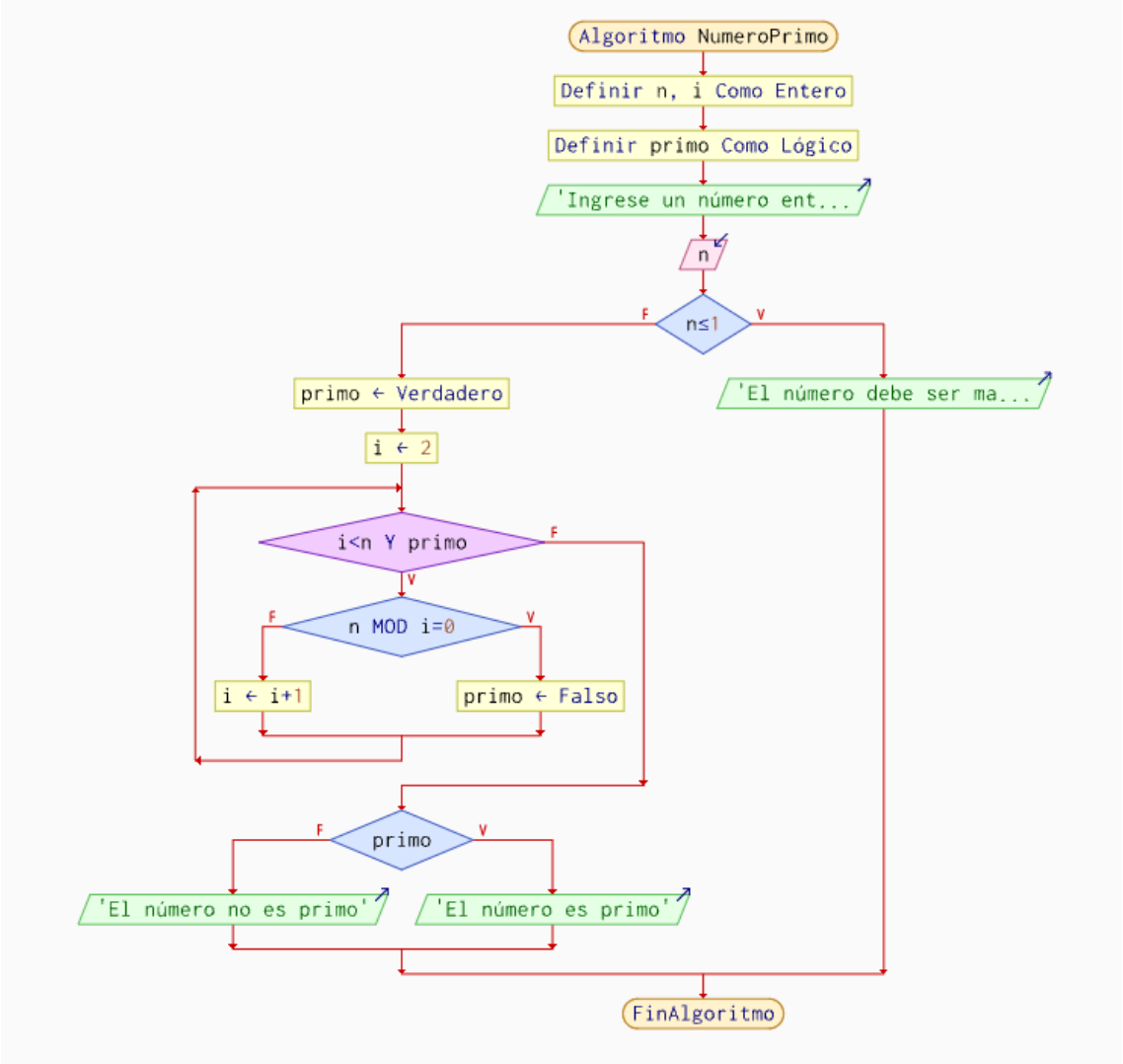
    Escribir "Ingrese un número entero mayor que 1:";
    Leer n;

    Si n ≤ 1 Entonces
        Escribir "El número debe ser mayor que 1";
    SiNo
        primo ← Verdadero;
        i ← 2;

        Mientras i < n Y primo Hacer
            Si n MOD i = 0 Entonces
                primo ← Falso;
            SiNo
                i ← i + 1;
            FinSi
        FinMientras

        Si primo Entonces
            Escribir "El número es primo";
        SiNo
            Escribir "El número no es primo";
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

i	n	primo	Condición (n<=1)(i<n Y primo)	salida
	5	verdadero	Falso	Ingrese un numero entero
2			Falso	
3				
4			Falso	
5			verdadero	

				El numero es primo

Código fuente

```c
#include <stdio.h>

int main()
{
    int n, i;
    int primo;

    // Lectura de datos por teclado
    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &n);

    // Validación inicial
    if (n <= 1) {
        printf("El numero debe ser mayor que 1\n");
    } else {
        primo = 1; // 1 equivale a Verdadero
        i = 2;

        // Bucle Mientras: evalúa i < n Y QUE el número siga siendo considerado primo
        while (i < n && primo == 1) {
            if (n % i == 0) { // El operador % calcula el resto o módulo
                primo = 0;    // Se encontró un divisor, por lo tanto NO es primo (Falso)
            } else {
                i = i + 1;
            }
        }

        // Evaluación final de la bandera
        if (primo == 1) {
            printf("El numero es primo\n");
        } else {
            printf("El numero no es primo\n");
        }
    }

    return 0;
}
```

Ejecución

```
Ingrese un numero entero mayor que 1: 5
El numero es primo

Process returned 0 (0x0)   execution time : 3.376 s
Press any key to continue.
```

Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

Análisis del problema

Entrada: Dos números enteros ingresados.

Proceso: Ordenar el rango, iterar con un ciclo externo desde el inicio hasta fin. Si el valor es menor o igual a 1, no es primo. Si el valor es mayor a 1, aplica la lógica de verificación de primos. Recorrer con un ciclo interno probando divisores i desde dos hasta el valor menos 1.

Salida: Lista de números primos encontrados entre 2 valores ingresados.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Primer limite	**a**	**Var**	**Entero**
Segundo limite	**b**	**Var**	**Entero**
Limite inferior	**inicio**	**Var**	**Entero**
Limite superior	**fn**	**Var**	**Entero**
Numero actual a evaluar	**num**	**Var**	**Entero**
Probador de divisores	**I**	**Var**	**Entero**
Indicador primo	**primo**	**Var**	**lógico**

Seudocódigo

```
Algoritmo PrimosRango
    Definir a, b, inicio, fn, num, i Como Entero;
    Definir esPrimo Como Logico;

    Escribir "Ingrese el primer número límite (a): ";
    Leer a;
    Escribir "Ingrese el segundo número límite (b): ";
    Leer b;

    Si a ≤ b Entonces
        inicio ← a;
    fn ← b;
SiNo
    inicio ← b;
fin ← a;
FinSi

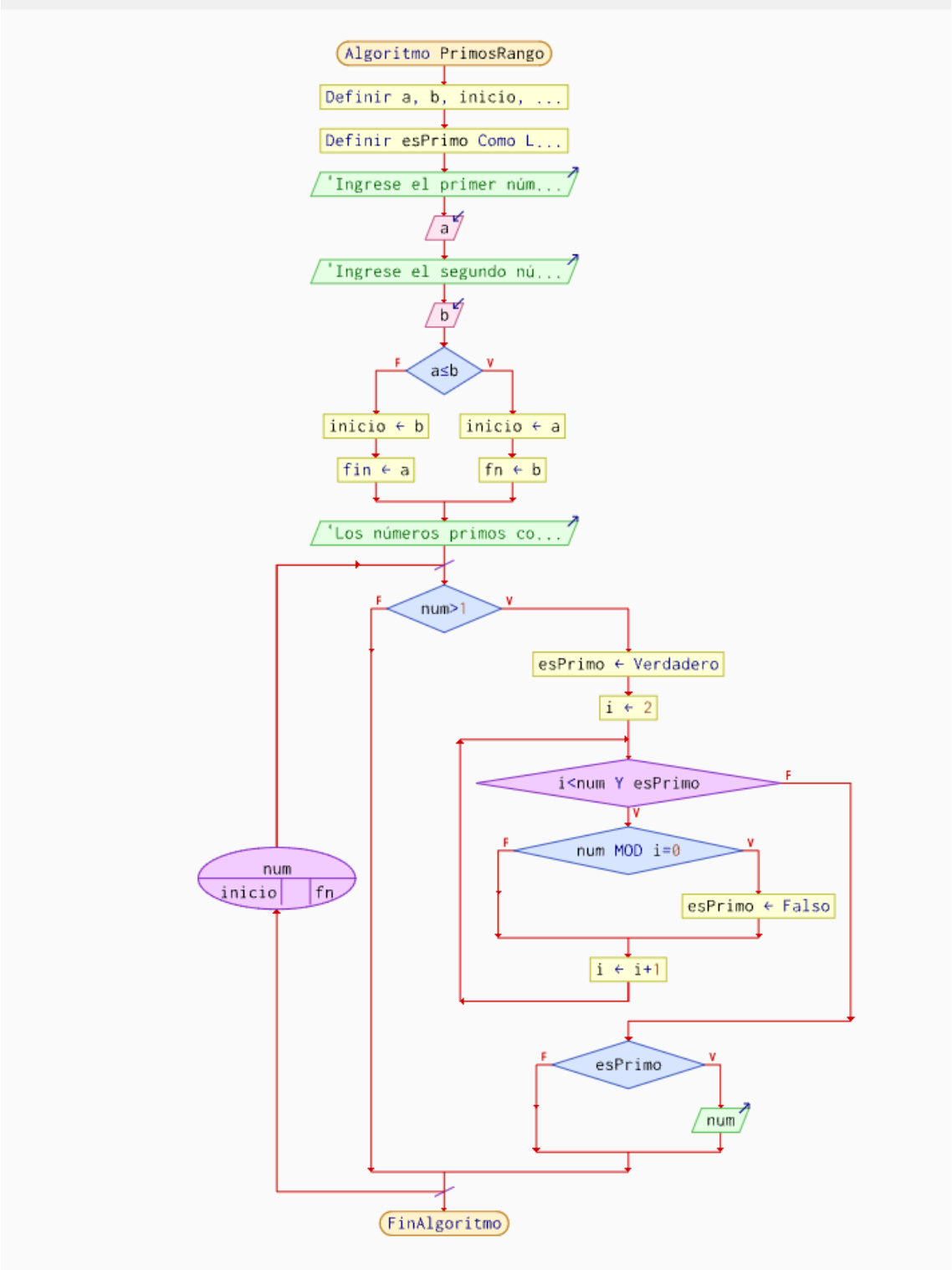
Escribir "Los números primos comprendidos entre ", inicio, " y ", fn, " son: ";


 Para num ← inicio Hasta fn Hacer
     Si num > 1 Entonces
        esPrimo ← Verdadero;
        i ← 2;

        Mientras i < num Y esPrimo Hacer
            Si num MOD i = 0 Entonces
                esPrimo ← Falso;
            FinSi
            i ← i + 1;
        FinMientras

        Si esPrimo Entonces
            Escribir num;
        FinSi
     FinSi
 FinPara
 FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

a	b	esPrimo	fn	i	inicio	num	Condicion	Salida
2					2			Ingrese el primer limite
	4		4	2		2		Ingrese el segundo limite
		Verdadero				3		
						4		
								Lo números primos entre el rango son: 2, 3

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables enteras
    int a, b, inicio, fin, num, i;
    int esPrimo; // 1 = Verdadero, 0 = Falso

    // Lectura de los límites
    printf("Ingrese el primer numero limite (a): ");
    scanf("%d", &a);
    printf("Ingrese el segundo numero limite (b): ");
    scanf("%d", &b);

    // Ordenamiento automático del rango (de menor a mayor)
    if (a <= b) {
        inicio = a;
        fin = b;
    } else {
        inicio = b;
        fin = a;
    }

    printf("\nLos numeros primos comprendidos entre %d y %d son:\n", inicio, fin);
```

```c
    // Bucle para recorrer cada número dentro del rango [inicio, fin]
    for (num = inicio; num <= fin; num++) {
        // Los números primos deben ser estrictamente mayores a 1
        if (num > 1) {
            esPrimo = 1; // Asumimos inicialmente que es primo
            i = 2;

            // Bucle Mientras para buscar divisores exactos
            while (i < num && esPrimo == 1) {
                if (num % i == 0) {
                    esPrimo = 0; // Se encontró un divisor -> No es primo
                }
                i = i + 1;
            }

            // Si al salir del bucle sigue siendo primo, lo mostramos
            if (esPrimo == 1) {
                printf("%d ", num);
            }
        }
    }

    printf("\n");
    return 0;
}
```

Ejecución

```
Ingrese el primer numero limite (a): 2
Ingrese el segundo numero limite (b): 4

Los numeros primos comprendidos entre 2 y 4 son:
2 3

Process returned 0 (0x0)   execution time : 2.333 s
Press any key to continue.
```

Problema 1.10. Factorial

Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1 · 2 · … · n, sabiendo que 0! = 1.

Análisis del problema

Entrada: Un número entero que sea mayor o igual a cero.

Proceso: Evaluar que el número sea mayor o igual que cero. Iniciar en la variable acumuladora del producto. Recorrer con un bucle para desde i igual a 1 hasta n. Si n es igual a cero, el bucle para nos ejecuta y la variable factorial se con conserva en 1.

Salida: Un mensaje con el resultado del factorial.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero de entrada	n	var	Entero
Iterador	i	var	Entero
acumulador	Factorial	var	Entero

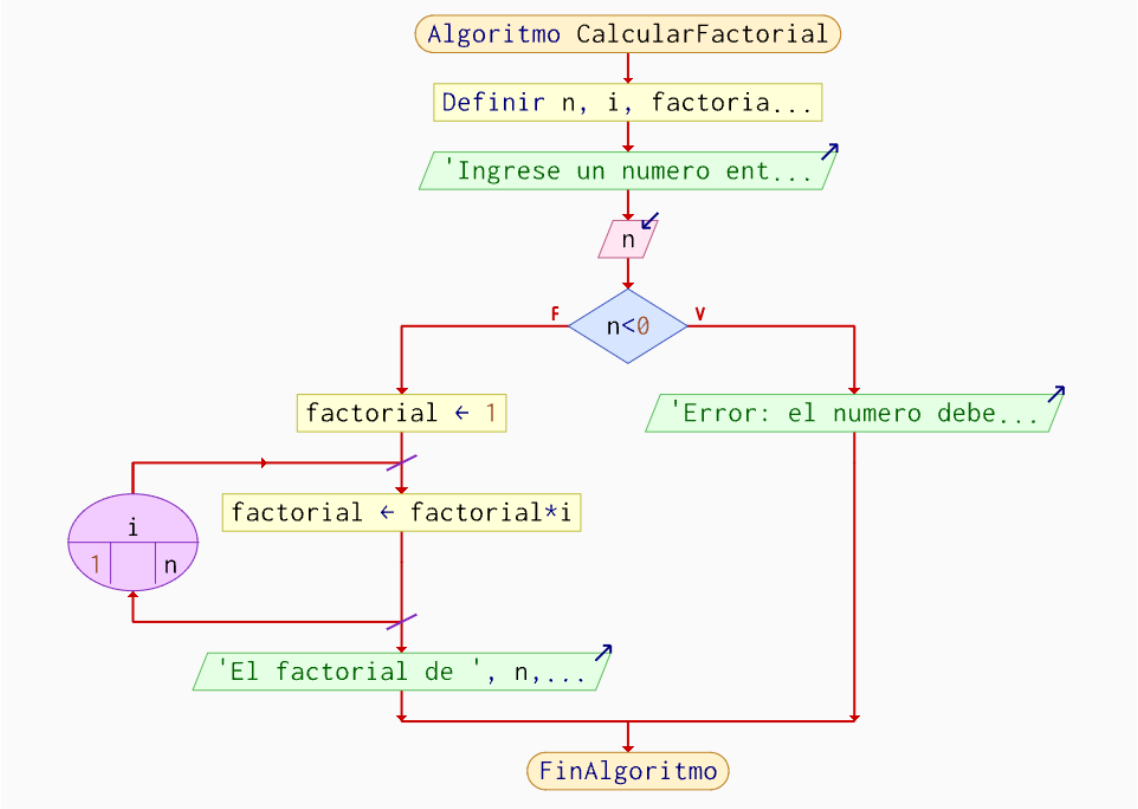
Seudocódigo

```
Algoritmo CalcularFactorial
    Definir n, i, factorial Como Entero;

    Escribir "Ingrese un numero entero mayor o igual a 0:";
    Leer n;

    Si n < 0 Entonces
        Escribir "Error: el numero debe ser mayor o igual a 0";
    SiNo
        factorial ← 1;
        Para i ← 1 Hasta n Hacer
            factorial ← factorial * i;
        FinPara
        Escribir "El factorial de ", n, " es: ", factorial;
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

factorial	i	n	Condición (n<0)	salida
		3	falso	Ingrese un numero entero mayor o igual cero
1	1		I=1	
1	2		I=2	
2	3		I=3	
6				
6				El factorial de 3 es 6

Código fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables enteras
    int n, i;
    long long factorial; // Usamos 'long long' porque los factoriales crecen muy rápido

    // Lectura de datos por teclado
    printf("Ingrese un numero entero mayor o igual a 0: ");
    scanf("%d", &n);

    // Validación inicial
    if (n < 0) {
        printf("Error: el numero debe ser mayor o igual a 0\n");
    } else {
        factorial = 1; // Inicialización de la identidad multiplicativa

        // Bucle Para (for) desde 1 hasta n
        for (i = 1; i <= n; i++) {
            factorial = factorial * i;
        }

        // Salida de datos (%lld para imprimir el tipo long long)
        printf("El factorial de %d es: %lld\n", n, factorial);
    }

    return 0;
}
```

Ejecución

```
Ingrese un numero entero mayor o igual a 0: 3
El factorial de 3 es: 6

Process returned 0 (0x0)   execution time : 0.755 s
Press any key to continue.
```

Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, …), donde cada término es la suma de los dos anteriores.

Análisis del problema.

Entrada: un número entero mayor o igual a 1.

Proceso: si el número es igual a 1 imprime únicamente cero. Si el número es mayor o igual a dos, inicializar a en cero y b en 1, imprimir a y b. Iterar con un bucle desde i igual a 3 hasta n calcular la siguiente<- a + b. Imprimir siguiente actualizar a y b menor o igual a siguiente.

Salida: los primeros términos de la serie de Fibonacci.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Términos a mostrar	n	Ingresado por teclado	Entero
Primer término anterior	a	0	Entero
Segundo término actual	b	1	Entero
Termino siguiente	siguiente	a+b	Entero
Iterador	i	Var	Entero

Seudocódigo

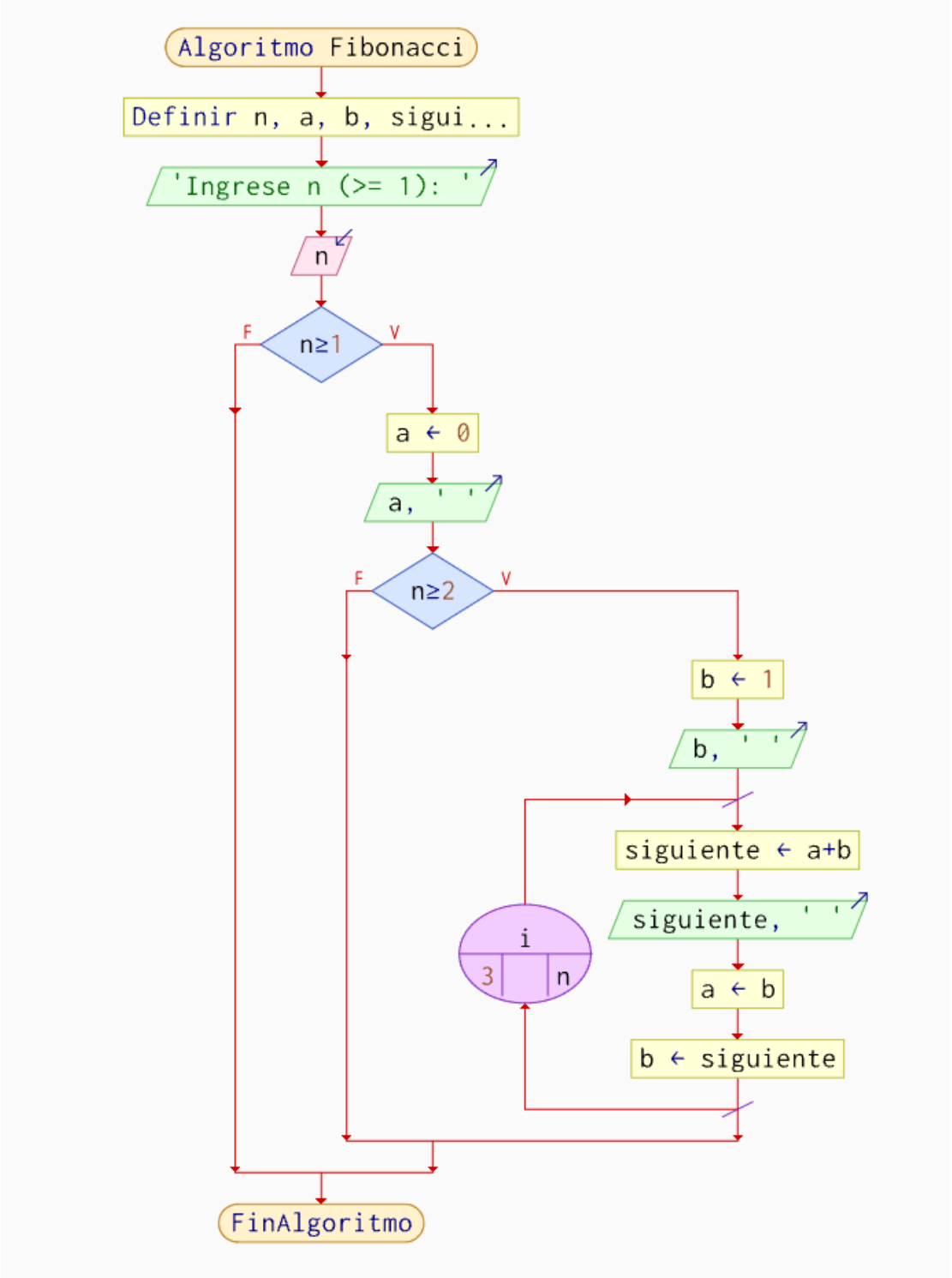
```
Algoritmo Fibonacci
    Definir n, a, b, siguiente, i Como Entero;
    Escribir "Ingrese n (>= 1): ";
    Leer n;

    Si n ≥ 1 Entonces
        a ← 0;
        Escribir Sin Saltar a, " ";

        Si n ≥ 2 Entonces
            b ← 1;
            Escribir Sin Saltar b, " ";

            Para i ← 3 Hasta n Hacer
                siguiente ← a + b;
                Escribir Sin Saltar siguiente, " ";
                a ← b;
                b ← siguiente;
            FinPara
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

n	a	b	siguiente	i	Condicion (n>=1)(n>=2)	Salida
4					Verdadero	Ingrese un numero
4	0	1			Verdadero	0
4	0	1	1	3	Verdadero	1
4	1	1	1	3		1
4	1	1	2	4	Verdadero	
4	1	2	2	4		2
4	1	2	2	5	Verdadero	

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables tipo entero
    int n, a, b, siguiente, i;

    // Lectura de datos por teclado
    printf("Ingrese n (>= 1): ");
    scanf("%d", &n);

    // Evaluación para imprimir el primer término
    if (n >= 1) {
        a = 0;
        printf("%d ", a);

        // Evaluación para imprimir el segundo término
        if (n >= 2) {
            b = 1;
            printf("%d ", b);

            // Bucle Para para generar a partir del tercer término en adelante
            for (i = 3; i <= n; i++) {
                siguiente = a + b;
                printf("%d ", siguiente);
                a = b;
                b = siguiente;
```

```
28              }
29          }
30      }
31
32      printf("\n");
33      return 0;
34  }
35
```

Ejecución

```
Ingrese n (>= 1): 4
0 1 1 2

Process returned 0 (0x0)   execution time : 3.544 s
Press any key to continue.
```

Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.

Análisis del problema

Entrada: El número entero positivo mayor es cero

proceso: iniciar la suma en cero. Y tras el número sean mayor a cero extraer el último dígito con dígito en n MOD 10 y acumular. Eliminar el último dígito mediante división entera.

Salida: La suma acumulada de dígitos.

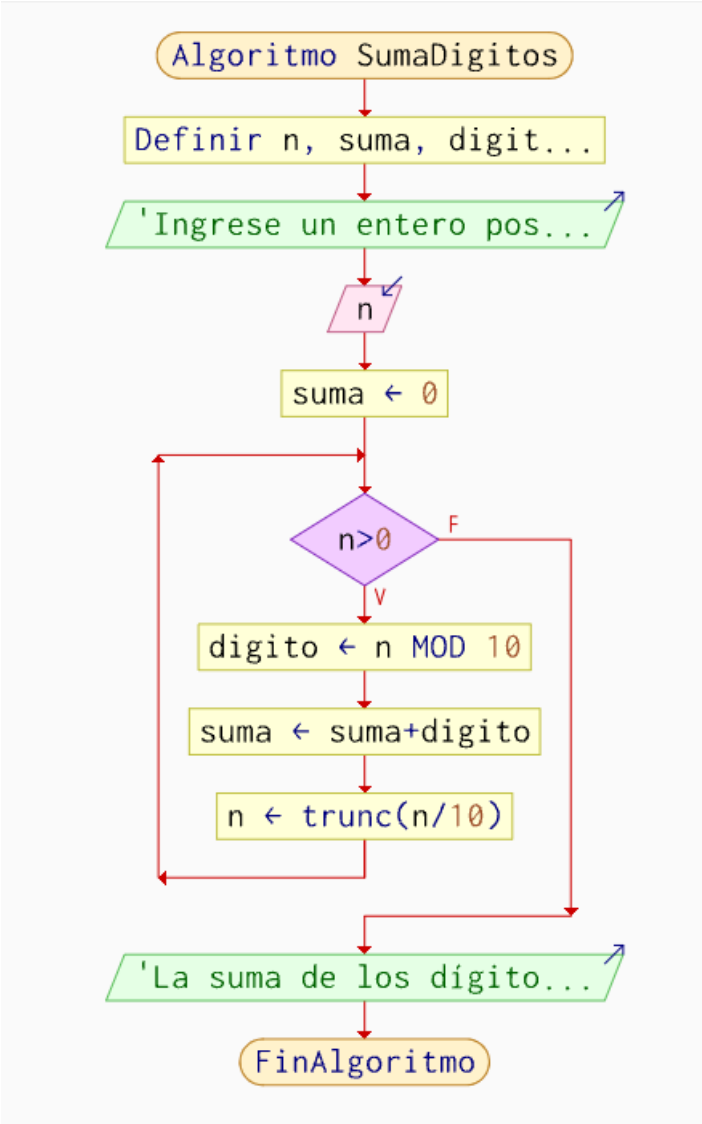
Tabla de Objetos

Objeto	**Nombre**	**Valor**	**Tipo**
Número a procesar	n	Ingresado por teclado	entero
Acumulador de la suma	suma	Suma de dígitos	entero
Ultimo digito extraído	digito	n MOD10	entero

Seudocódigo

```
Algoritmo SumaDigitos
    Definir n, suma, digito Como Entero;
    Escribir "Ingrese un entero positivo: ";
    Leer n;
    suma ← 0;
    Mientras n > 0 Hacer
        digito ← n MOD 10;
        suma ← suma + digito;
        n ← trunc(n / 10);
    FinMientras
    Escribir "La suma de los dígitos es ", suma;
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

digito	n	suma	Condición n>0	salida
	42	0	Verdadero	Ingrese un numero
2	4	2	Verdadero	
4	4	6	Verdadero	
4	0	6	Falso	La suma de los dígitos es 6

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables enteras
    int n, suma, digito;

    // Entrada de datos
    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    // Inicialización del acumulador de suma
    suma = 0;

    // Bucle Mientras: se ejecuta mientras queden dígitos por extraer (n > 0)
    while (n > 0) {
        digito = n % 10;      // extrae el último dígito usando el operador módulo (%)
        suma = suma + digito; // acumula el dígito extraído
        n = n / 10;           // al dividir entre enteros en C, se descarta automáticamente la parte decimal (truncado)
    }

    // Salida de datos
    printf("La suma de los digitos es %d\n", suma);

    return 0;
}
```

Ejecución

```
Ingrese un entero positivo: 42
La suma de los digitos es 6

Process returned 0 (0x0)   execution time : 4.803 s
Press any key to continue.
```

Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

Análisis del problema

Entrada: Un numero entero positivo.

Proceso: Respaldar el numero original. Mediante un ciclo, construir el numero al revés. Comparar con el controlador Si.

Salida: Mensaje que indica si es capicúa o no.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número a evaluar	n	Ingresado por teclado	Entero
Respaldo original	original	n	Entero

Copia de trabajo	aux	n	Entero
Numero invertido	inveritdo	Acumulador invertido	Entero

Seudocódigo

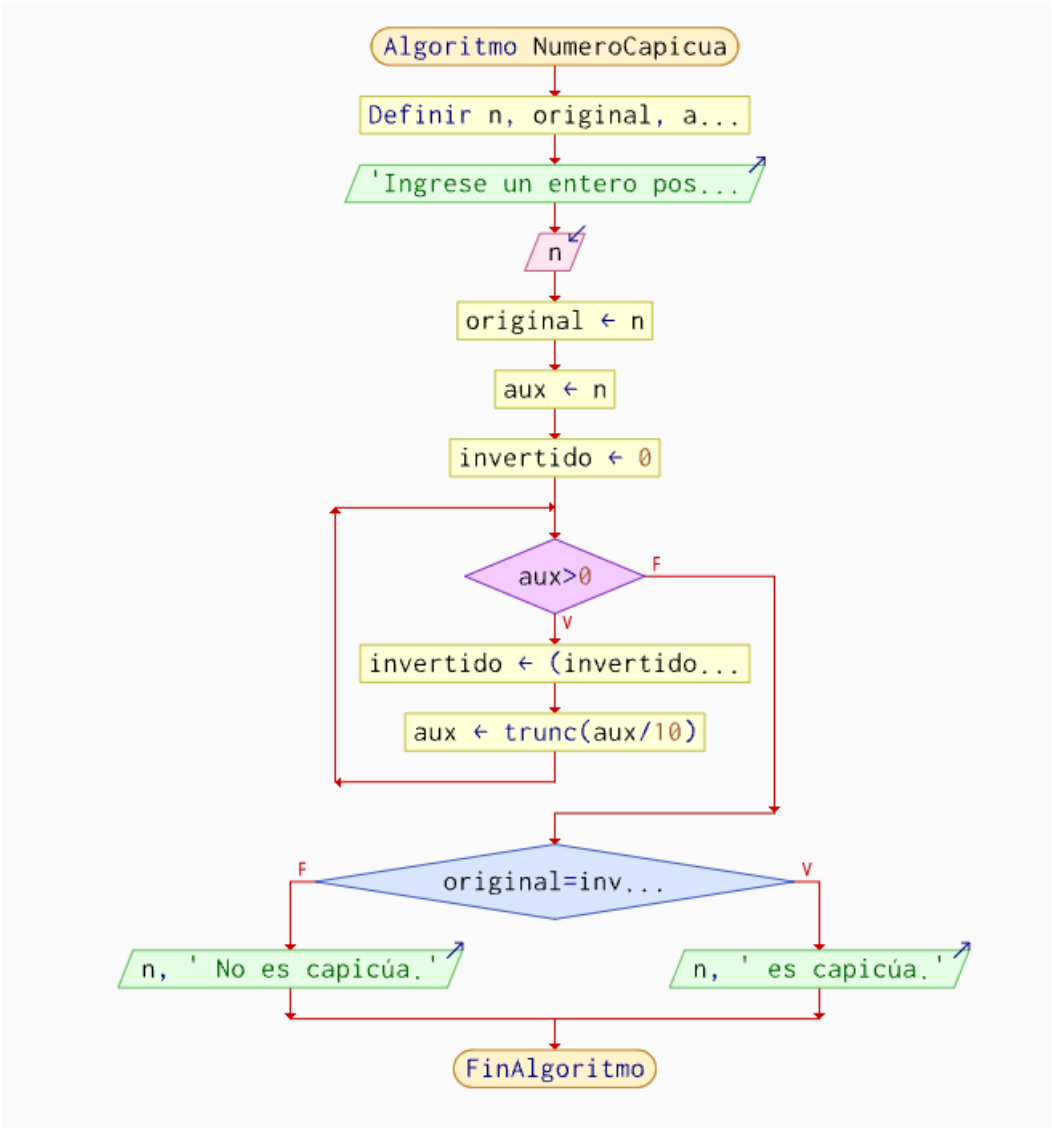
```
Algoritmo NumeroCapicua
    Definir n, original, aux, invertido Como Entero;
    Escribir "Ingrese un entero positivo: ";
    Leer n;

    original ← n;
    aux ← n;
    invertido ← 0;

    Mientras aux > 0 Hacer
        invertido ← (invertido * 10) + (aux MOD 10);
        aux ← trunc(aux / 10);
    FinMientras

    Si original = invertido Entonces
        Escribir n, " es capicúa.";
    SiNo
        Escribir n, " No es capicúa.";
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

aux	invertido	n	original	Condición aux>0	Salida
121	0	121	121	verdadero	Ingreso un entero positivo
12	1	121	121	verdadero	
1	12	121	121	verdadero	
0	121	121	121	falso	

0	**121**	**121**	**121**	**verdadero**	**121 es capicua**

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables tipo entero
    int n, original, aux, invertido;

    // Entrada de datos
    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    // Guardamos copias del número original para comparar al final
    original = n;
    aux = n;
    invertido = 0;

    // Bucle Mientras para construir el número invertido dígito a dígito
    while (aux > 0) {
        invertido = (invertido * 10) + (aux % 10);
        aux = aux / 10; // La división de enteros en C descarta automáticamente los decimales
    }

    // Comprobación de capicúa
    if (original == invertido) {
        printf("%d es capicua.\n", n);
    } else {
        printf("%d No es capicua.\n", n);
    }

    return 0;
```

Ejecución

```
Ingrese un entero positivo: 121
121 es capicua.

Process returned 0 (0x0)   execution time : 1.751 s
Press any key to continue.
```

Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD.

Análisis del problema

Entrada: Dos números enteros positivos (a y b).

Proceso: Mientras b sea diferente de 0 guardar temporalmente b en temp. Calcular el residuo b<- a MOD b y asignar a<- temp. Al terminar el bucle cuando b=0, el MCD se encuentra en a,

Salida: El MCD de a.

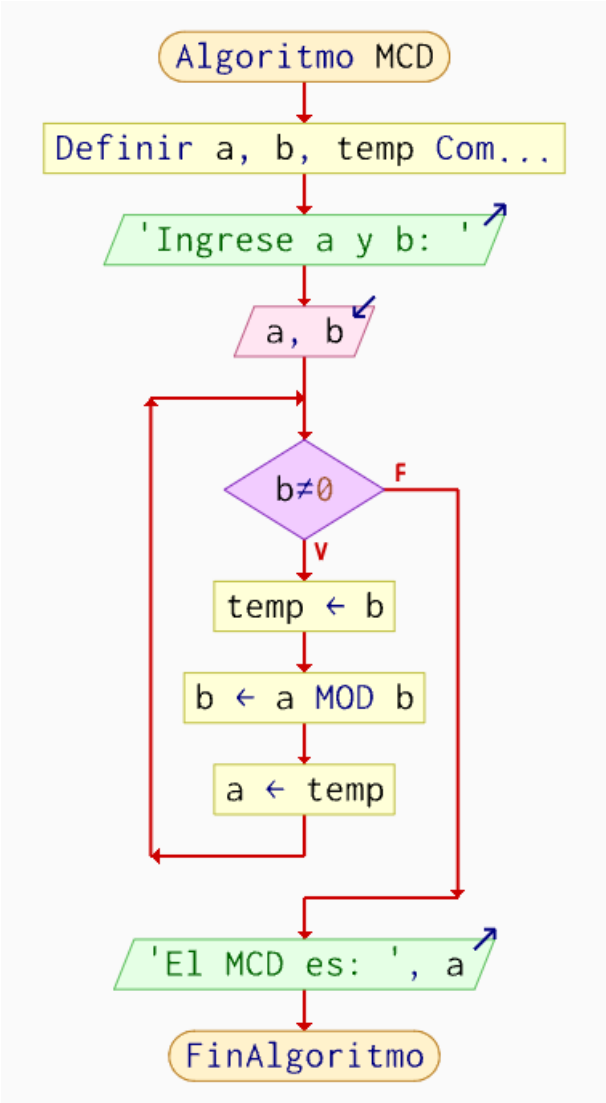
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Primer numero	a	Ingresado por teclado	Entero
Residuo	b	Ingresado por teclado	Entero
Variable temporal	temp	Análisis de intercambio	Entero

Seudocódigo

```
Algoritmo MCD
    Definir a, b, temp Como Entero;
    Escribir "Ingrese a y b: ";
    Leer a, b;
    Mientras b ≠ 0 Hacer
        temp ← b;
        b ← a MOD b;
        a ← temp;
    FinMientras
    Escribir "El MCD es: ", a;
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

a	b	Condición b<>0	Salida
48	18	Verdadero	Ingrese a y b
18	12	Verdadero	
12	6	Verdadero	
6	0	Falso	El MCD es 6

Código fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables enteras
    int a, b, temp;

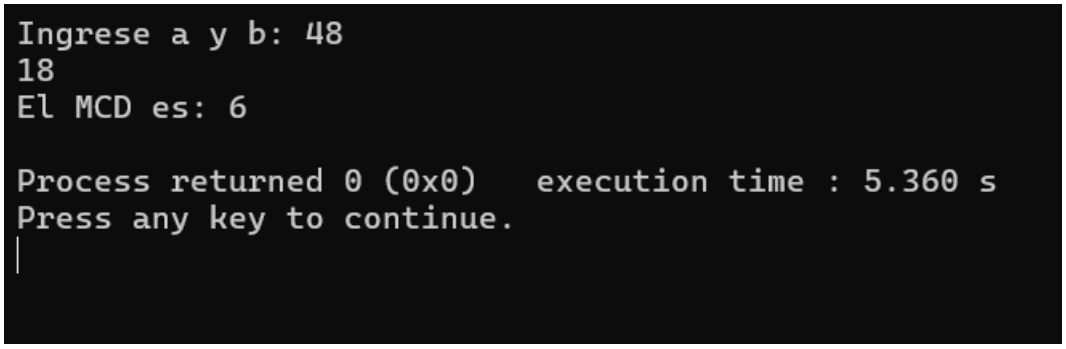
    // Entrada de datos
    printf("Ingrese a y b: ");
    scanf("%d %d", &a, &b);

    // Bucle Mientras (Algoritmo de Euclides)
    while (b != 0) {
        temp = b;         // Guardamos el valor actual de b
        b = a % b;        // Calculamos el residuo/módulo entre a y b
        a = temp;         // Actualizamos 'a' con el antiguo valor de 'b'
    }

    // Salida del Máximo Común Divisor
    printf("El MCD es: %d\n", a);

    return 0;
}
```

Ejecución



Problema 1.15. Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

Análisis del problema

Entrada: Un año entero positivo.

Proceso: Un año bisiesto si cumple (anio MOD 4==0 Y anio MOD 100<>0).

Salida: Mensaje de confirmación.

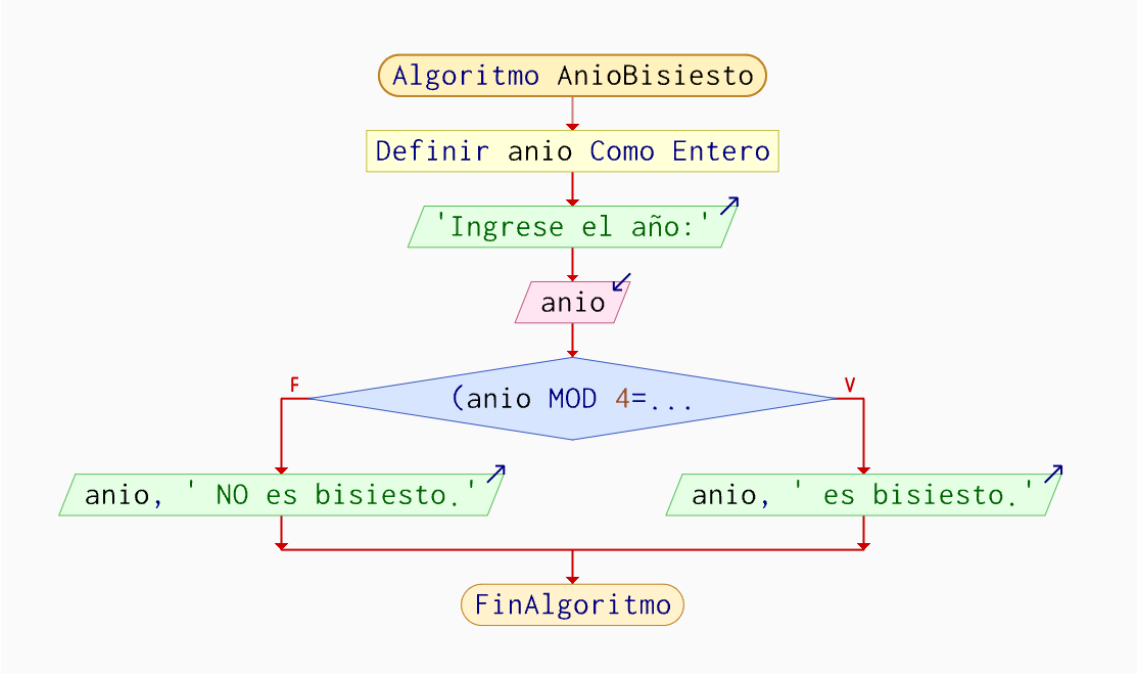
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Año a consultar	anio	Ingresado por teclado	entero

Seudocódigo

```
Algoritmo AnioBisiesto
    Definir anio Como Entero;
    Escribir "Ingrese el año:";
    Leer anio;
    Si (anio MOD 4 = 0 Y anio MOD 100 ≠ 0) O (anio MOD 400 = 0) Entonces
        Escribir anio, " es bisiesto.";
    SiNo
        Escribir anio, " NO es bisiesto.";
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

anio	Condicion	Evaluado	Salida
2024	**2024%4==0 Y 2024%100<>0**	**Verdadero**	**Ingrese un año**

2021		**Verdadero**	**2024 es bisiesto**

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variable entera
    int anio;

    // Entrada de datos
    printf("Ingrese el anio: ");
    scanf("%d", &anio);

    // Condicional compuesto con operadores lógicos:
    // && equivale a Y (AND)
    // != equivale a <> (Diferente de)
    // || equivale a O (OR)
    if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
        printf("%d es bisiesto.\n", anio);
    } else {
        printf("%d NO es bisiesto.\n", anio);
    }

    return 0;
}
```

Ejecución

```
Ingrese el anio: 2024
2024 es bisiesto.

Process returned 0 (0x0)   execution time : 2.229 s
Press any key to continue.
```

Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

Análisis del problema

Entrada: Un numero entero positivo mayor a cero

Proceso: Iniciar la suma<-0. Iterar con d desde 1 hasta n-1. Si n MOD d==0, acumular suma<- suma + d. Comparar si suma==n

Salida: Mensaje que indica si es perfecto o no

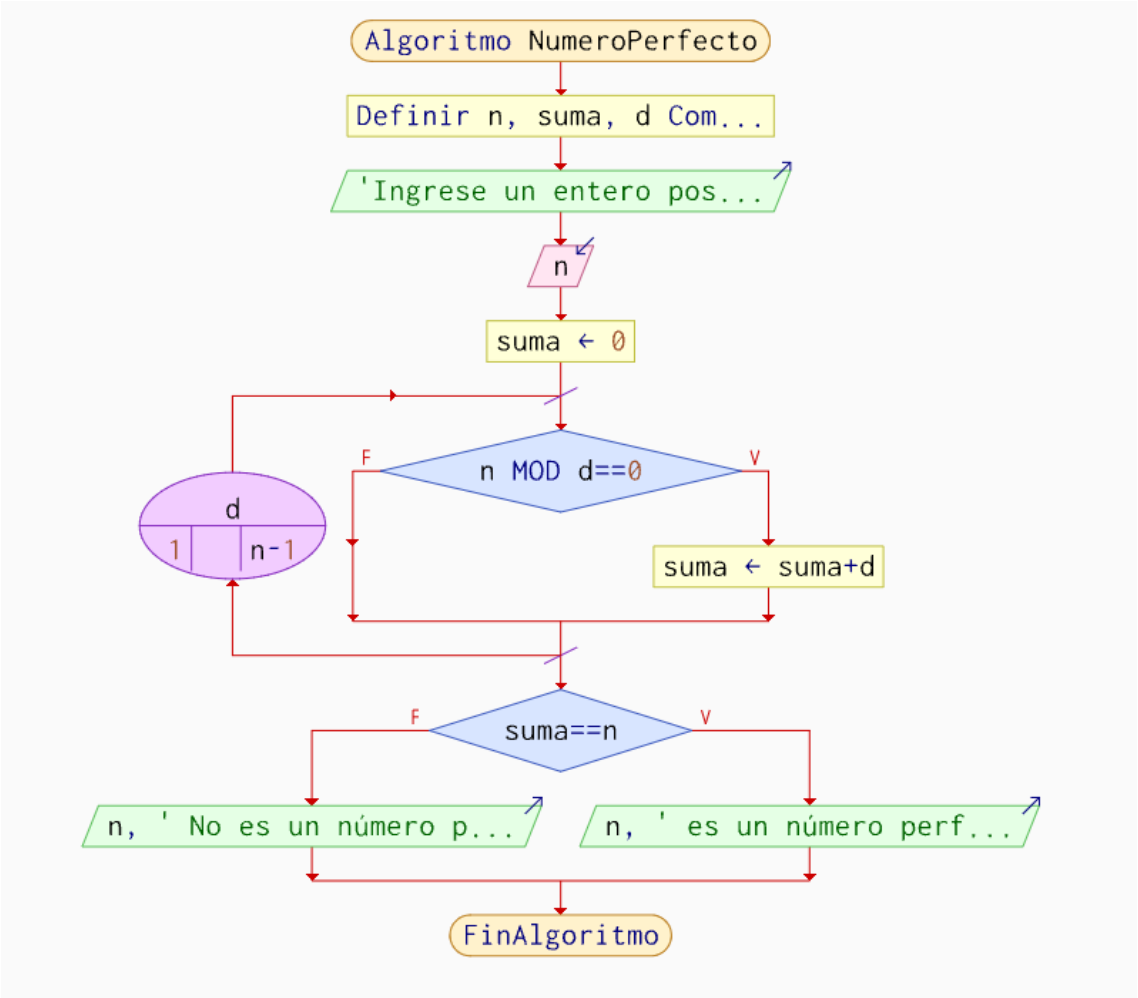
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Numero a evaluar	n	Ingresado por teclado	Entero
Suma de divisores	suma	Acumulador de divisores	Entero
Posible divisor	d	Iterador de 1 a n-1	Entero

Seudocódigo

```
1  Algoritmo NumeroPerfecto
2      Definir n, suma, d Como Entero;
3      Escribir "Ingrese un entero positivo:";
4      Leer n;
5      suma ← 0;
6      Para d ← 1 Hasta n - 1 Hacer
7          Si n MOD d == 0 Entonces
8              suma ← suma + d;
9          FinSi
10     FinPara
11     Si suma == n Entonces
12         Escribir n, " es un número perfecto.";
13     SiNo
14         Escribir n, " No es un número perfecto.";
15     FinSi
16 FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

n	d	n%d==0	suma	Suma==n	Salida
6	1	Verdadero	1	verdadero	Ingrese un numero positivo
6	2	Verdadero	3	verdadero	
6	3	Verdadero	6	verdadero	
6	4	Verdadero	6	verdadero	
6	5	Falso	6	verdadero	
6	5	Falso	6	verdadero	6 es un numero perfecto

Código fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables de tipo entero
    int n, suma, d;

    // Lectura del número por teclado
    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    // Inicialización del acumulador
    suma = 0;

    // Bucle para buscar divisores propios desde 1 hasta n - 1
    for (d = 1; d <= n - 1; d++) {
        // En C, el operador módulo (MOD) es el símbolo de porcentaje %
        if (n % d == 0) {
            suma = suma + d; // Acumulamos el divisor hallado
        }
    }

    // Verificación final si la suma de divisores es igual al número original
    if (suma == n) {
        printf("%d es un numero perfecto.\n", n);
    } else {
        printf("%d No es un numero perfecto.\n", n);
    }

    return 0;
```

Ejecución

```
Ingrese un entero positivo: 6
6 es un numero perfecto.

Process returned 0 (0x0)   execution time : 1.463 s
Press any key to continue.
```

Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

Análisis del problema

Entrada: Un número entero positivo en base 10(n>0)

Proceso: Iniciar la cadena binaria. Mientras n>0, extraer el residuo y concatenando por la izquierda la cadena existente para invertir el orden automáticamente sin usar arreglos. Dividir n entre 2.

Salida: La representación binaria en formato de texto.

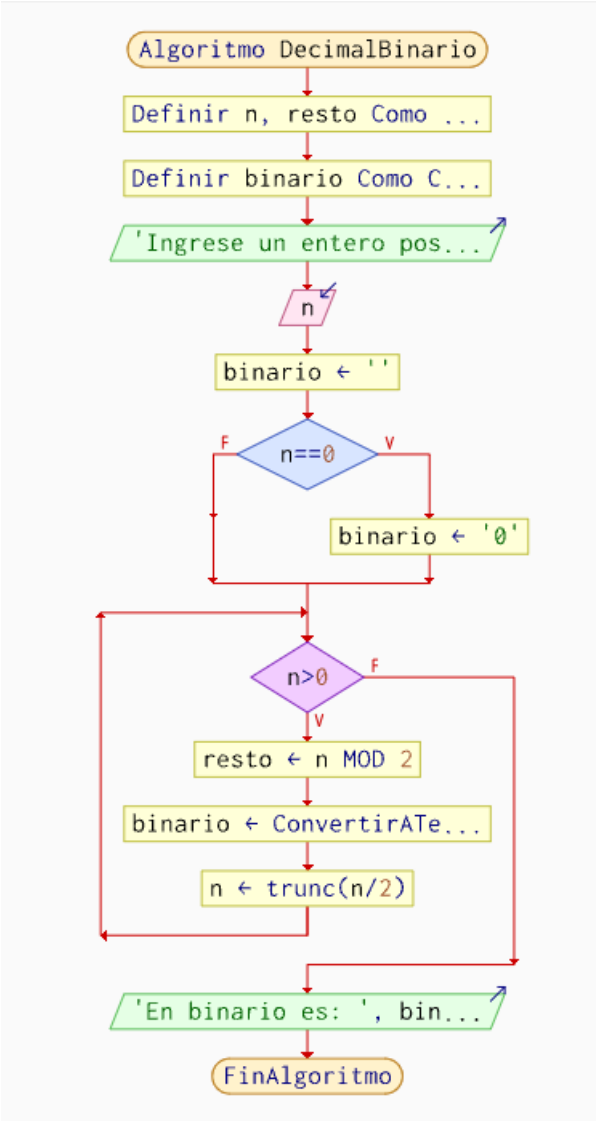
Tabla de objeto

Objeto	nombre	Valor	tipo
Numero decimal	n	Ingresado por teclado	Entero
Residuo de division	Resto	n mod 2	Entero
Cadena binaria	binario	Resultado acumulado	Cadena de texto

Seudocódigo

```
Algoritmo DecimalBinario
    Definir n, resto Como Entero;
    Definir binario Como Cadena;
    Escribir "Ingrese un entero positivo:";
    Leer n;
    binario ← "";
    Si n == 0 Entonces
        binario ← "0";
    FinSi
    Mientras n > 0 Hacer
        resto ← n MOD 2;
        binario ← ConvertirATexto(resto) + binario;
        n ← trunc(n / 2);
    FinMientras
    Escribir "En binario es: ", binario;
FinAlgoritmo
```

DIAGRAMA DE FLUJO



Prueba de escritorio

n	resto	binario	n>0	Salida
5		“”	Verdadero	Ingresa un numero
2	1	“1”	Verdadero	
1	0	“01”	Verdadero	
0	1	“101”	Falso	En binario es 101

Código Fuente

```c
#include <stdio.h>

int main()
{
    // Declaración de variables tipo entero
    int n, resto;
    int binario[32]; // Arreglo para almacenar los dígitos binarios (hasta 32 bits)
    int i = 0;

    // Entrada de datos por teclado
    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

    // Caso especial para el número 0
    if (n == 0) {
        printf("En binario es: 0\n");
    } else {
        // Bucle Mientras: extraer residuos de divisiones sucesivas entre 2
        while (n > 0) {
            resto = n % 2;          // Operador módulo en C (%)
            binario[i] = resto;     // Guardamos el residuo en el arreglo
            i++;
            n = n / 2;              // División entera que trunca los decimales
        }

        // Imprimir el resultado invirtiendo el orden de los residuos guardados
        printf("En binario es: ");
        for (int j = i - 1; j >= 0; j--) {
            printf("%d", binario[j]);
        }
        printf("\n");
    }

    return 0;
}
```

Ejecución

```
Ingrese un entero positivo: 5
En binario es: 101

Process returned 0 (0x0)   execution time : 1.290 s
Press any key to continue.
```

Problema 1.18. Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.

Análisis del problema

Entrada: Un entero mayor o igual a 1.

Proceso: Usar un ciclo externo para las filas de 1 a n. Usar un ciclo interno para las columnas de 1 a final para imprimir. Imprimir un salto de línea al terminar cada fila.

Salida: Patron de asteriscos de forma de triángulo rectángulo.

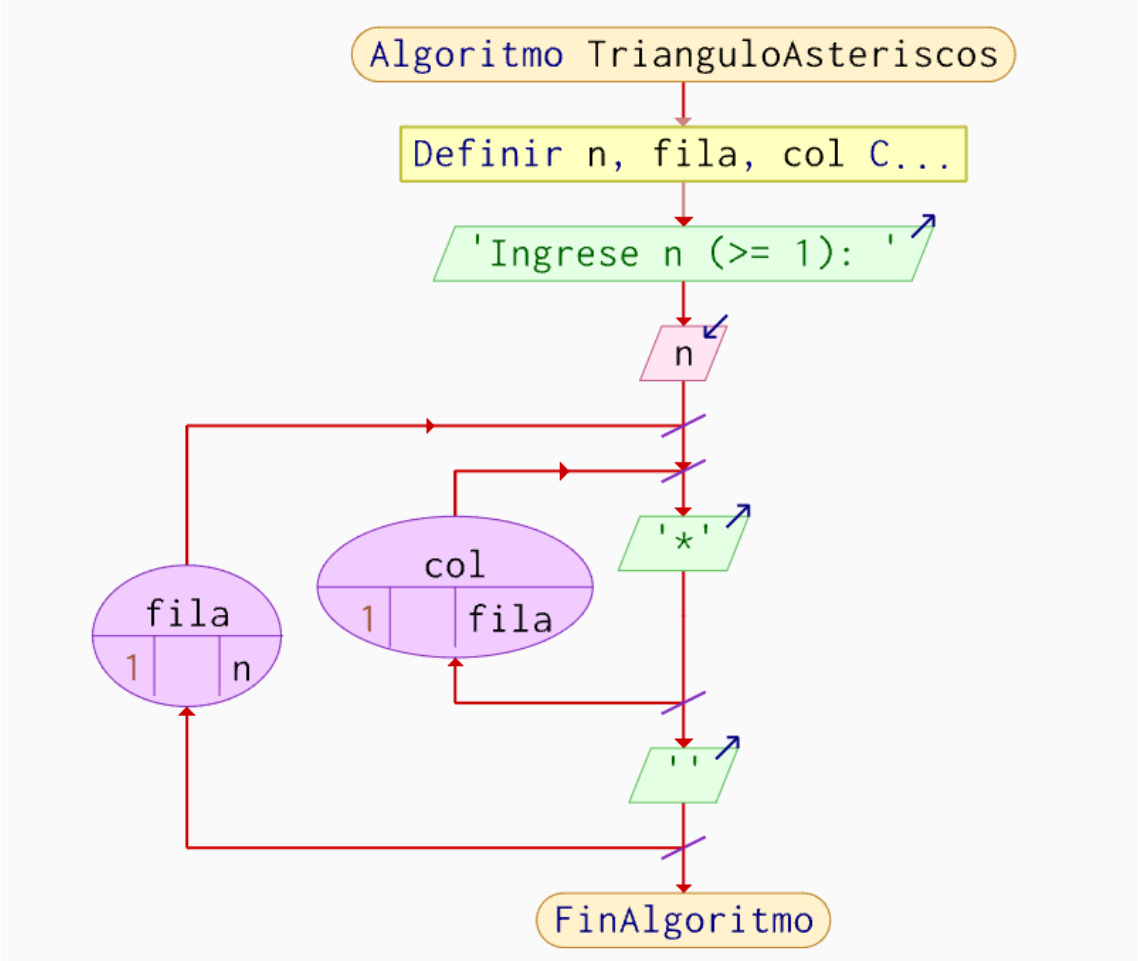
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Filas a dibujar	**n**	**Ingresado por teclado**	**Entero**
Control de fila actual	**fila**	**1,2,….,n**	**Entero**
Control de asteriscos	**col**	**1,2,….,n**	**Entero**

Seudocódigo

```
1  Algoritmo TrianguloAsteriscos
2      Definir n, fila, col Como Entero;
3      Escribir "Ingrese n (>= 1): ";
4      Leer n;
5      Para fila ← 1 Hasta n Hacer
6          Para col ← 1 Hasta fila Hacer
7              Escribir Sin Saltar "*";
8          FinPara
9          Escribir "";
10     FinPara
11 FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

n	fila	col	col>=fila	Salida
3	1	1	Verdadero	*
3	1	2	Falso	
3	2	1	Verdadero	* **
3	2	2	Verdadero	
3	2	3	Falso	***
3	3	1,2,3	Verdadero	
3	3	4	Falso	

Código fuente

```c
#include <stdio.h>

int main()
{
    int n, fila, col;

    printf("Ingrese n (>= 1): ");
    scanf("%d", &n);

    // Bucle externo para recorrer las filas
    for (fila = 1; fila <= n; fila++) {
        // Bucle interno para imprimir asteriscos por cada columna
        for (col = 1; col <= fila; col++) {
            printf("*");
        }
        printf("\n"); // Salto de línea al terminar cada fila
    }

    return 0;
}
```

Ejecución

```
Ingrese n (>= 1): 5
*
**
***
****
*****

Process returned 0 (0x0)   execution time : 1.087 s
Press any key to continue.
```

Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

Análisis de problema

Entrada: secuencia de enteros positivos que finaliza al ingresar el valor 0.

Proceso: Mantener las variables mayor<-0, segundo <-0 y contador<-0. Al recibir num>0. Si num>mayor, desplazar segundo<-mayor y actualizar mayor<-num. De lo contrario, si num>segundo, actualizar segundo<-num. Al ingresar 0, verificar >=2.

Salida: Mostrar mayor y segundo, o informar si hubo menos de 2 valores.

Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número leído	**num**	**Ingresado por teclado**	**Entero**
Número mayor	**mayor**	**Valor máximo**	**Entero**
Segundo mayor	**segundo**	**Segundo máximo**	**Entero**
Cantidad de datos	**contador**	**Total ingresado**	**Entero**

Seudocódigo

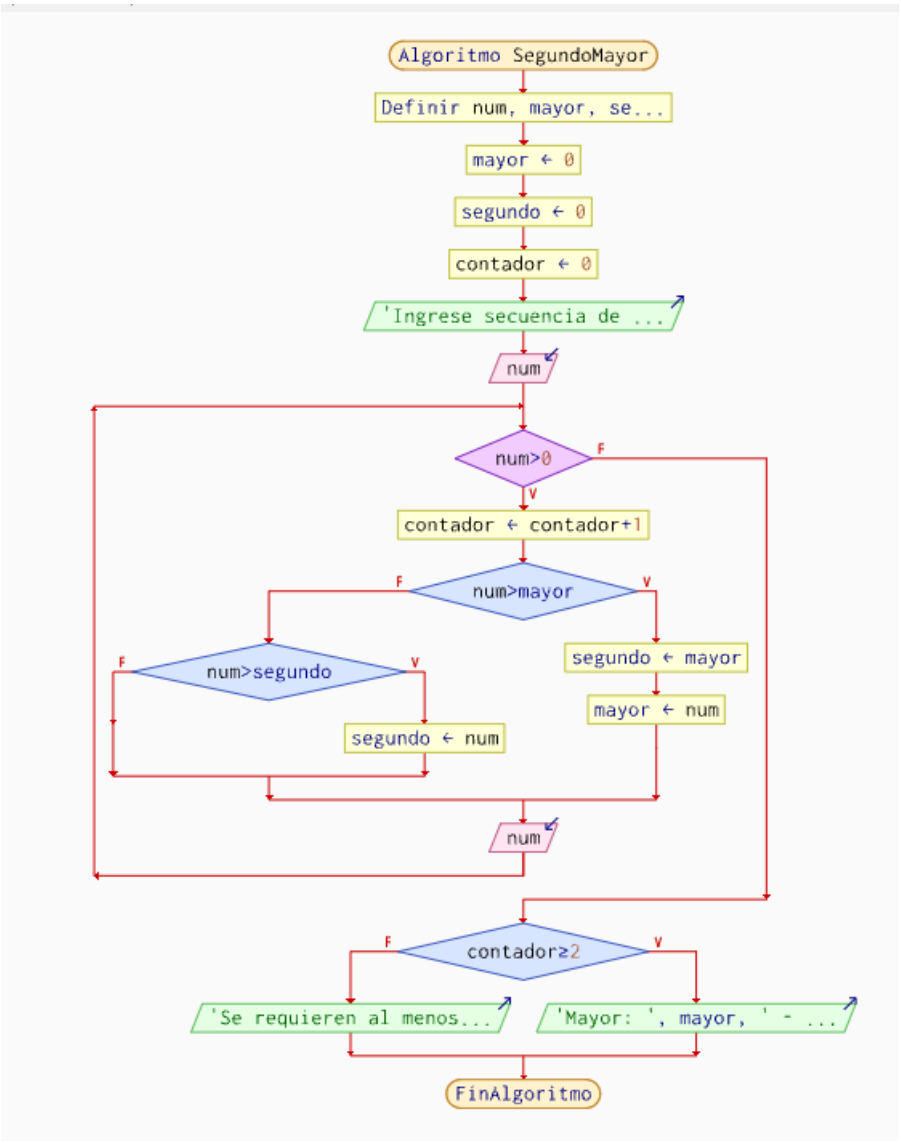
```
Algoritmo SegundoMayor
    Definir num, mayor, segundo, contador Como Entero;
    mayor ← 0;
    segundo ← 0;
    contador ← 0;

    Escribir "Ingrese secuencia de números positivos (0 para terminar): ";
    Leer num;

    Mientras num > 0 Hacer
        contador ← contador + 1;
        Si num > mayor Entonces
            segundo ← mayor;
            mayor ← num;
        SiNo
            Si num > segundo Entonces
                segundo ← num;
            FinSi
        FinSi
        Leer num;
    FinMientras

    Si contador ≥ 2 Entonces
        Escribir "Mayor: ", mayor, " - Segundo mayor: ", segundo;
    SiNo
        Escribir "Se requieren al menos dos números en la secuencia.";
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

num	mayor	segundo	contador	(num>0) (num>mayor) (num>=segundo) (contador>=2)	Salida
					Ingrese una secuencia

					de numeros
7	7	0	1	Verdadero	
9	9	7	2	Verdadero	
0	9	7	2	Falso	
0	9	7	2	Falso	Mayor:9 Segundo mayor:7

Código Fuente

```c
#include <stdio.h>

int main()
{
    int num, mayor = 0, segundo = 0, contador = 0;

    printf("Ingrese secuencia de numeros positivos (0 para terminar): ");
    scanf("%d", &num);

    while (num > 0) {
        contador++;
        if (num > mayor) {
            segundo = mayor; // El antiguo mayor pasa a ser el segundo mayor
            mayor = num;     // El número nuevo ocupa el primer lugar
        } else if (num > segundo) {
            segundo = num;   // Se actualiza únicamente el segundo mayor
        }
        scanf("%d", &num);
    }

    if (contador >= 2) {
        printf("Mayor: %d - Segundo mayor: %d\n", mayor, segundo);
    } else {
        printf("Se requieren al menos dos numeros en la secuencia.\n");
    }

    return 0;
}
```

Ejecución

```
Ingrese secuencia de numeros positivos (0 para terminar): 1956
46123
8945
654198
0
Mayor: 654198 - Segundo mayor: 46123

Process returned 0 (0x0)   execution time : 6.569 s
Press any key to continue.
```

Problema 1.20. Adivina el número

Desarrolla un programa que “piense” un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar “más alto”, “más bajo” o “¡acertaste!”, y al final mostrar cuántos intentos se usaron.

Análisis del problema

Entrada: Intentos numéricos del usuario (intento) hasta acertar con el número secreto (42).

Proceso: definir el número secreto e iniciar el contador. Repetir la lectura en un bucle, incrementar. Si intento<secreto, mostar “mas alto”; si intento>secreto, mostrar “mas bajo”. Repetir hasta que intento==secreto.

Salida: Mensaje “ACERTASTE” y el total de intentos utilizados.

Tabla de objetos

Objeto	**Nombre**	**Valor**	**Tipo**
Número a adivinar	secreto	Valor fijo (42)	Entero
Intento del usuario	Intento	Ingresado por teclado	Entero
Contador de intentos	intentos	Acumulador de pruebas	Entero

Seudocódigo

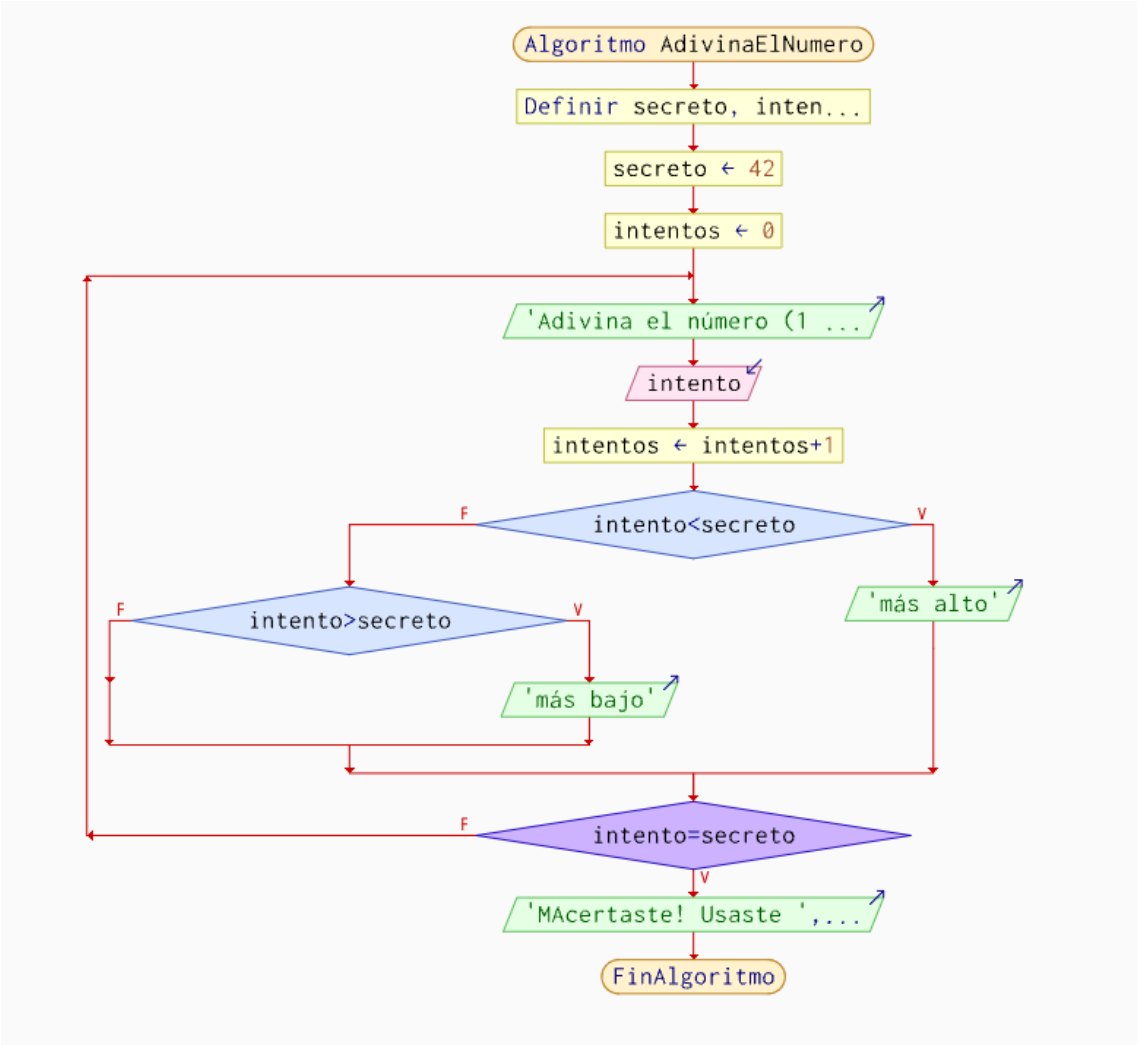
```
Algoritmo AdivinaElNumero
    Definir secreto, intento, intentos Como Entero;
    secreto ← 42;
    intentos ← 0;

    Repetir
        Escribir "Adivina el número (1 al 100): ";
        Leer intento;
        intentos ← intentos + 1;

        Si intento < secreto Entonces
            Escribir "más alto";
        SiNo
            Si intento > secreto Entonces
                Escribir "más bajo";
            FinSi
        FinSi
    Hasta Que intento = secreto

    Escribir "¡Acertaste! Usaste ", intentos, " intentos.";
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

intento	secreto	intentos	condición	Salida
50				**50**
	42	**1**	**50>42**	**Mas bajo**
42	**42**	**1**	**42==42**	**Acertaste (2 intentos)**

Código Fuente

```c
#include <stdio.h>

int main()
{
    int secreto = 42, intento, intentos = 0;

    // En C, el bucle Repetir-Hasta Que equivale a do-while
    do {
        printf("Adivina el numero (1 al 100): ");
        scanf("%d", &intento);
        intentos++;

        if (intento < secreto) {
            printf("mas alto\n");
        } else if (intento > secreto) {
            printf("mas bajo\n");
        }
    } while (intento != secreto); // Continúa MIENTRAS NO sea igual al secreto

    printf("¡Acertaste! Usaste %d intentos.\n", intentos);

    return 0;
}
```

Ejecución

```
Adivina el numero (1 al 100): 50
mas bajo
Adivina el numero (1 al 100): 42
íAcertaste! Usaste 2 intentos.

Process returned 0 (0x0)   execution time : 7.077 s
Press any key to continue.
```